



UNIVERSITÀ DEGLI STUDI DI PADOVA

Corso di Laurea in Informatica
Insegnamento di Ingegneria del Software

Anno Accademico 2025/2026

Norme di Progetto

Regole, Standard e Way of Working

Gruppo: Synergo Unit

Email: synergounit.20@gmail.com

Versione: **v2.14.0**

Data ultima modifica: **2026-08-01**

Registro Delle Modifiche

Versione	Data	Autore	Verificatore	Descrizione
2.14.0	2026-08-01	Armando Moda Scarati	Anton R. Rupì	Transizione a PB: aggiornate convenzioni di codifica, struttura repository, PR policy e DoD.
2.13.0	2026-07-27	Anton R. Rupì	Roberto M. Doroftei	Aggiornamento processo di sviluppo.
2.12.0	2026-06-09	Sofia De Blasi	Armando M. Scarati	Inserimento gestione issue aggiornamento sito.
2.11.0	2026-05-30	Sofia De Blasi	Roberto M. Doroftei	Aggiornamento metriche di qualità.
2.10.1	2026-05-28	Sofia De Blasi	Roberto M. Doroftei	Correzione refusi.
2.10.0	2026-05-28	Sofia De Blasi	Roberto M. Doroftei	Aggiornati workflow repository, issue, branch, commit, PoC e metriche di qualità.
2.9.0	2026-05-24	Sofia De Blasi	Roberto M. Doroftei	Correzione refusi.
2.8.0	2026-05-24	Sofia De Blasi	Roberto M. Doroftei	Aggiornati processi introduzione e tailoring.
2.7.0	2026-05-24	Sofia De Blasi	Roberto M. Doroftei	Aggiornati processi organizzativi.
2.6.0	2026-05-23	Sofia De Blasi	Roberto M. Doroftei	Aggiornati processi di supporto.
2.5.0	2026-05-23	Sofia De Blasi	Roberto M. Doroftei	Aggiornati processi primari.
2.4.0	2026-05-07	Sofia De Blasi	Francesco Marcon	Aggiornati processi organizzativi.
2.3.0	2026-05-06	Sofia De Blasi	Francesco Marcon	Aggiornati processi di supporto.
2.2.0	2026-05-05	Sofia De Blasi	Francesco Marcon	Aggiornati processi primari.
2.1.0	2026-05-05	Sofia De Blasi	Francesco Marcon	Aggiornato tailoring.
2.0.0	2026-05-05	Responsabile: Anton R. Rupì		Approvazione ingresso in baseline RTB.
1.1.2	2026-03-25	Sofia De Blasi	Francesco Marcon	Aggiornata email.
1.1.1	2026-03-22	Sofia De Blasi	Filippo Idiometri	Sistemata punteggiatura.

Versione	Data	Autore	Verificatore	Descrizione
1.1.0	2026-03-21	Sofia De Blasi	Filippo Idiometri	Aggiunti pedici G.
1.0.0	2026-03-21	Responsabile: Sofia De Blasi		Approvazione ingresso in baseline CC.
0.5.0	2026-03-20	Sofia De Blasi	Filippo Idiometri	Ristrutturazione secondo ISO/IEC 12207:1995; aggiunta sezione Tailoring.
0.4.0	2026-03-20	Roberto M. Doroftei	Sofia De Blasi	Stesura sezione Tipologia di Lavoro: modalità decisionale e brainstorming.
0.3.0	2026-03-19	Sofia De Blasi	Francesco Marcon	Stesura sezione Strumenti di Progetto: gestione, documentazione e comunicazione.
0.2.0	2026-03-18	Sofia De Blasi	Francesco Marcon	Stesura sezione Documentazione: tipologie, convenzioni e versionamento.
0.1.0	2026-03-13	Roberto M. Doroftei	Sofia De Blasi	Stesura sezione Introduzione: scopo, glossario e riferimenti.
0.0.0	2026-03-13	Armando Moda Scarati	Sofia De Blasi	Prima stesura della struttura del documento.

Indice

Registro Delle Modifiche	1
1 Introduzione	9
1.1 Scopo del Documento	9
1.2 Scopo del Prodotto	9
1.3 Glossario	10
1.4 Convenzioni Tipografiche	10
1.5 Riferimenti	10
1.5.1 Normativi	10
1.5.2 Informativi	10
1.6 Organizzazione del Documento	11
2 Tailoring	12
2.1 Processi Attivati nella Fase CC	12
2.2 Processi Attivati nella Fase RTB	13
2.3 Processi Attivi nella Fase PB	13
3 Processi Primari	15
3.1 Processo di Fornitura	15
3.1.1 Descrizione	15
3.1.2 Norme e Strumenti del Processo di Fornitura	16
3.1.2.1 Strumenti a Supporto	16
3.1.2.2 Documentazione prodotta in fase CC	16
3.1.2.3 Documentazione prodotta o prevista in fase RTB	17
3.1.3 Attività del Processo di Fornitura	19
3.1.3.1 Inizializzazione	19
3.1.3.2 Risposta	19
3.1.3.3 Consolidamento degli impegni	19
3.1.3.4 Pianificazione	20
3.1.3.5 Esecuzione e controllo	20
3.1.3.6 Revisione	20
3.1.3.7 Validazione esterna	20
3.1.3.8 Consegna e completamento	21
3.2 Processo di Sviluppo	21
3.2.1 Descrizione	21
3.2.2 Implementazione del processo	22

3.2.3	Analisi dei Requisiti di sistema	22
3.2.3.1	Scopo	22
3.2.3.2	Raccolta dei requisiti	23
3.2.3.3	Classificazione	23
3.2.3.4	Gestione delle modifiche	23
3.2.4	Progettazione dell'architettura di sistema	23
3.2.4.1	Scopo	23
3.2.5	Analisi dei requisiti software	23
3.2.5.1	Scopo	23
3.2.5.2	Nomenclatura dei requisiti	23
3.2.5.3	Tracciabilità	24
3.2.5.4	Gestione dei Casi d'Uso	24
3.2.5.5	Convenzioni dei casi d'uso	25
3.2.6	Progettazione dell'architettura software	25
3.2.6.1	Scopo	25
3.2.6.2	Integrazione dello Stato dell'Arte e selezione tecnologica	25
3.2.6.3	Stile Architetturale (Fase PB)	26
3.2.7	Progettazione dettagliata del software	26
3.2.7.1	Scopo	26
3.2.7.2	Gestione delle Dipendenze (Dependency Injection)	26
3.2.8	Codifica e test del software	26
3.2.8.1	Scopo	26
3.2.8.2	Perimetro del PoC	27
3.2.8.3	Tecnologie del PoC	27
3.2.8.4	Criteri di successo	27
3.2.8.5	Validazione del PoC	28
3.2.8.6	Sviluppo del Prodotto (Fase PB)	28
3.2.8.7	Convenzioni di Nomenclatura (Naming)	28
3.2.9	Integrazione del software	28
3.2.9.1	Scopo	28
3.2.9.2	Workflow Git e CI/CD	29
3.2.9.3	Test di Integrazione (TI)	29
3.2.10	Test di qualifica del software	29
3.2.10.1	Scopo	29
3.2.10.2	Mappatura UC-Test	29
3.2.10.3	Metriche di Prodotto	29
3.2.11	Integrazione del sistema	29

3.2.11.1	Scopo	29
3.2.11.2	Interazione con attori esterni	29
3.2.11.3	Containerizzazione	29
3.2.12	Test di qualifica del sistema	29
3.2.12.1	Scopo	29
3.2.12.2	Validazione della Technology Baseline	29
3.2.13	Installazione del software	30
3.2.13.1	Scopo	30
3.2.13.2	Manuale Tecnico (README)	30
3.2.14	Supporto all'accettazione del software	30
3.2.14.1	Scopo	30
3.2.14.2	Validazione con la Proponente	30
4	Processi di Supporto	31
4.1	Processo di Documentazione	31
4.1.1	Descrizione	31
4.1.2	Tipologia dei Documenti	31
4.1.2.1	Documentazione Gestionale Interna	31
4.1.2.2	Documentazione Gestionale Esterna	32
4.1.2.3	Documentazione Tecnica	32
4.1.3	Strumenti di Documentazione	32
4.1.4	Convenzioni Redazionali	33
4.1.5	Template Documentale	34
4.1.6	Registro delle Modifiche	34
4.1.7	Gestione del Glossario	35
4.1.8	Checklist Prima della Pull Request	35
4.1.9	Ciclo di Vita Documentale _G	35
4.1.10	Versionamento Documentale	36
4.2	Processo di Gestione della Configurazione	36
4.2.1	Descrizione	36
4.2.2	Sistema di Gestione	37
4.2.3	Struttura del Repository	37
4.2.4	Gestione delle Issue	38
4.2.4.1	Issue documentali	38
4.2.4.2	Issue non documentali	38
4.2.4.3	Campi obbligatori	39
4.2.5	Strategia di Branching	39

4.2.5.1	Branch documentali	40
4.2.5.2	Branch non documentali	40
4.2.6	Commit	41
4.2.6.1	Scope Documentali	42
4.2.6.2	Esempi	42
4.2.7	Pull Request	42
4.2.8	Branch Protection Rules	43
4.2.9	Merge	43
4.2.10	Generazione e Pubblicazione dei PDF	43
4.2.11	Gestione del Sito Documentale	43
4.3	Processo di Verifica	44
4.3.1	Descrizione	44
4.3.2	Responsabilità	44
4.3.3	Attività di Verifica	45
4.3.4	Verifica del Codice del Proof of Concept	45
4.3.5	Verifica del Codice (Analisi Statica e Testing)	46
4.3.6	Esito della Verifica	46
4.3.7	Definition of Done	46
4.3.7.1	Criteri Comuni	46
4.3.7.2	Criteri per Task Documentali	47
4.3.7.3	Criteri per Task di Implementazione del PoC	47
4.3.7.4	Criteri per Task di Sviluppo (Fase PB)	48
4.4	Processo di Validazione	48
4.4.1	Descrizione	48
4.4.2	Validazione con la Proponente	48
4.5	Processo di Accertamento della Qualità	49
4.5.1	Descrizione	49
4.5.2	Modello di Miglioramento	49
4.5.3	Metriche e Piano di Qualifica	49
4.5.3.1	Metriche di qualità di processo	50
4.5.3.2	Metriche di qualità di prodotto	50
4.6	Processo di Gestione dei Cambiamenti	51
4.6.1	Descrizione	51
4.6.2	Tipologie di Cambiamento	51
4.6.3	Procedura	51
4.6.4	Responsabilità di Approvazione	52
4.7	Processo di Gestione delle Non Conformità	52

4.7.1	Descrizione	52
4.7.2	Classificazione	52
4.7.3	Gestione	53
5	Processi Organizzativi	54
5.1	Processo di Gestione	54
5.1.1	Descrizione	54
5.1.2	Composizione del Gruppo	54
5.1.3	Ruoli del Gruppo	54
5.1.4	Costi Orari dei Ruoli	55
5.1.5	Responsabilità Operative Specifiche	56
5.1.6	Pianificazione degli Sprint	56
5.1.7	Gestione dei Task Non Completati	57
5.1.8	Tracciamento Ore e Costi	57
5.1.9	Gestione dei Rischi	58
5.1.10	Riunione del Martedì	58
5.1.11	Riunione del Venerdì	59
5.1.12	Daily Stand-Up Asincrono	59
5.1.13	Verbali e Tracciamento Decisioni	59
5.2	Processo di Comunicazione	60
5.2.1	Descrizione	60
5.2.2	Comunicazione Interna	60
5.2.3	Comunicazione Esterna	61
5.2.4	Incontri con la Proponente	61
5.2.5	Comunicazione con il Committente	62
5.3	Processo di Infrastruttura	62
5.3.1	Descrizione	62
5.3.2	Responsabilità	62
5.3.3	Strumenti di Gestione	62
5.3.4	Strumenti di Documentazione	63
5.3.5	Strumenti di Comunicazione	63
5.3.6	Strumenti Tecnici	63
5.3.7	Infrastruttura del Proof of Concept	64
5.3.8	Struttura del Repository di Sviluppo	64
5.3.8.1	Frontend (Vue.js / TypeScript)	64
5.3.8.2	Backend (FastAPI / Python)	65
5.3.9	Adozione di Nuovi Strumenti	65

5.4	Processo di Miglioramento	65
5.4.1	Descrizione	65
5.4.2	Identificazione delle Criticità	65
5.4.3	Azioni Correttive	66
5.4.4	Aggiornamento del Way of Working	66
5.5	Processo di Formazione	66
5.5.1	Descrizione	66
5.5.2	Modalità di Formazione	67
5.5.3	Formazione su Strumenti di Processo	67
5.5.4	Formazione Tecnica	67
5.5.5	Applicazione della Formazione	68
A	Glossario	69

1 Introduzione

1.1 Scopo del Documento

Il presente documento definisce le norme, le convenzioni e le procedure operative adottate dal gruppo *Synergo Unit* nello svolgimento del progetto didattico del corso di Ingegneria del Software.

Esso costituisce il riferimento operativo interno del gruppo e ha lo scopo di garantire uniformità, tracciabilità e qualità durante l'intero ciclo di vita del progetto.

Le norme qui descritte sono organizzate in conformità allo standard **ISO/IEC 12207:1995_G**, applicato secondo il processo di tailoring_G descritto nella Sezione 2.

Le Norme di Progetto_G hanno i seguenti obiettivi:

- definire il *way of working_G* del gruppo;
- standardizzare le attività svolte;
- garantire tracciabilità e verificabilità degli artefatti prodotti;
- assicurare uniformità documentale e organizzativa;
- supportare il miglioramento continuo del processo;
- favorire una collaborazione strutturata tra i membri del gruppo.

Il documento è soggetto ad aggiornamento incrementale secondo il principio *just-in-time_G*: ogni nuova attività deve essere normata prima della sua adozione operativa e ogni evoluzione significativa del progetto può comportare l'introduzione di nuovi processi o l'estensione di quelli esistenti.

Le presenti norme prevalgono su eventuali convenzioni implicite o pratiche informali adottate dal gruppo.

Tutti i membri del gruppo sono tenuti a leggere e rispettare il presente documento.

1.2 Scopo del Prodotto

Il prodotto sviluppato consiste in una piattaforma documentale intelligente basata su tecnologie AI e Large Language Models (LLM_G), finalizzata al supporto della scrittura e della gestione di note Markdown_G.

La piattaforma dovrà supportare funzionalità di:

- gestione documentale Markdown;
- supporto AI alla scrittura;
- generazione di suggerimenti contestuali;
- interazione con Large Language Models;
- gestione strutturata delle note;
- interfaccia utente_G moderna, accessibile e responsive.

1.3 Glossario

I termini tecnici, gli acronimi e le espressioni potenzialmente ambigue utilizzate all'interno della documentazione sono definiti nel documento [Glossario](#).

La prima occorrenza dei termini presenti nel glossario viene marcata automaticamente tramite pedice_G mediante uno script Python dedicato descritto nella Sezione 4.1.3.

1.4 Convenzioni Tipografiche

All'interno della documentazione vengono adottate le seguenti convenzioni tipografiche:

- i termini inglesi e le espressioni tecniche vengono riportati in *corsivo*;
- i termini definiti nel Glossario sono marcati con il pedice_G alla prima occorrenza;
- i riferimenti a file, branch_G, comandi e identificativi tecnici vengono riportati tramite font monospace mediante il comando `\texttt{}`;
- i riferimenti interni alla documentazione vengono riportati tramite hyperlink.

1.5 Riferimenti

1.5.1 Normativi

- [Capitolato C6 – Second Brain \(Zucchetti S.p.A.\)](#);
- [Regolamento del Progetto Didattico – Corso di Ingegneria del Software, A.A. 2025/2026](#);
- [Standard ISO/IEC 12207:1995 – Information Technology – Software Life Cycle Processes](#).

1.5.2 Informativi

- Materiale didattico del corso di Ingegneria del Software, A.A. 2025/2026;
- [L^AT_EX Project Documentation](#);
- [GitHub Documentation](#);
- [Piano di Progetto_G, v1.18.1](#);
- [Piano di Qualifica_G, v1.3.0](#);
- [Analisi dei Requisiti_G, v1.0.0](#);
- [Analisi dello Stato dell'Arte_G, v1.0.0](#);
- Verbali interni ed esterni del progetto Second Brain.

1.6 Organizzazione del Documento

Il presente documento è organizzato nelle seguenti sezioni:

- **Tailoring:** descrive l'adattamento dei processi adottato dal gruppo rispetto al contesto progettuale.
- **Processi Primari:** descrive i processi direttamente coinvolti nello sviluppo del prodotto software;
- **Processi di Supporto:** descrive i processi che supportano lo sviluppo, la qualità, la documentazione e la gestione della configurazione_G;
- **Processi Organizzativi:** descrive le modalità organizzative adottate dal gruppo;

2 Tailoring

Lo standard ISO/IEC 12207:1995 prevede la possibilità di adattare (*tailoring*) i processi del ciclo di vita software al contesto specifico del progetto.

Il gruppo *Synergo Unit* adotta un approccio incrementale orientato al miglioramento continuo, introducendo progressivamente processi e pratiche in funzione dell'evoluzione del progetto e delle baseline_G previste dal Regolamento del Progetto Didattico.

Il tailoring adottato tiene conto dei seguenti fattori contestuali:

- **Baseline attuale:** PB_G; il gruppo si trova nella fase di sviluppo dei requisiti, delle scelte tecnologiche e della documentazione tecnica e gestionale richiesta per la Requirements & Technology Baseline;
- **Dimensione del gruppo:** sei membri con rotazione periodica e documentata dei ruoli;
- **Natura didattica del progetto:** il progetto viene svolto in ambito accademico, con vincoli temporali, organizzativi ed economici definiti dal regolamento del corso;
- **Approccio incrementale:** il gruppo introduce esclusivamente processi effettivamente utilizzati, verificabili e coerenti con lo stato corrente del progetto;
- **Principio just-in-time:** ogni nuova attività o pratica viene normata prima della sua applicazione operativa.

Il presente documento è soggetto ad aggiornamento progressivo: ogni evoluzione significativa del way of working, degli strumenti o delle attività di progetto comporta, se necessario, l'aggiornamento delle presenti Norme di Progetto.

2.1 Processi Attivati nella Fase CC

La seguente tabella riporta i processi attivati durante la fase di candidatura_G (CC), mantenuti operativi anche nelle fasi successive del progetto.

Categoria	Processo	Sezione
Primario	Processo di Fornitura	3.1
Di supporto	Processo di Documentazione	4.1
Di supporto	Processo di Gestione della Configurazione	4.2
Di supporto	Processo di Verifica	4.3
Organizzativo	Processo di Gestione	5.1
Organizzativo	Processo di Comunicazione	5.2
Organizzativo	Processo di Infrastruttura	5.3

2.2 Processi Attivati nella Fase RTB

Con l'avanzamento alla baseline RTB vengono introdotti ulteriori processi e attività necessari al consolidamento dei requisiti, alla motivazione delle scelte tecnologiche e all'implementazione dimostrativa del Proof of Concept_G.

Il Proof of Concept è finalizzato a dimostrare la fattibilità tecnica delle funzionalità principali individuate per la Requirements & Technology Baseline; non costituisce ancora il prodotto completo previsto per la Product Baseline_G.

I processi precedentemente attivati rimangono operativi.

Categoria	Processo	Sezione
Primario	Processo di Sviluppo: Analisi dei Requisiti	3.2
Primario	Processo di Sviluppo: Analisi dello Stato dell'Arte	3.2
Primario	Processo di Sviluppo: Progettazione Architettuale Preliminare	3.2
Primario	Processo di Sviluppo: Implementazione del Proof of Concept	3.2
Di supporto	Processo di Validazione _G	4.4
Di supporto	Processo di Accertamento della Qualità	4.5
Di supporto	Processo di Gestione dei Cambiamenti	4.6
Di supporto	Processo di Gestione delle Non Conformità _G	4.7
Organizzativo	Gestione dei Rischi	5.1
Organizzativo	Processo di Miglioramento	5.4
Organizzativo	Processo di Formazione	5.5

2.3 Processi Attivi nella Fase PB

Con il superamento della RTB e l'ingresso nella Product Baseline, il gruppo ha consolidato l'architettura e avviato lo sviluppo esteso del prodotto (Minimum Viable Product).

I processi precedentemente attivati rimangono operativi, e vengono formalmente normati e introdotti i seguenti processi per supportare lo sviluppo del codice di produzione:

- codifica estesa del prodotto e definizione dell'architettura;
- norme di codifica e naming conventions;
- Dependency Injection;
- test automatici (Unit e Integration testing);
- analisi statica del codice (Linting e Formattazione);
- Continuous Integration tramite GitHub Actions;

- gestione rigorosa delle Pull Request.

L'introduzione di questi processi si riflette nei corrispondenti capitoli dei Processi Primari, Organizzativi e di Supporto.

3 Processi Primari

In conformità allo standard ISO/IEC 12207:1995, i processi primari comprendono le attività direttamente coinvolte nella realizzazione, nella fornitura e nell'evoluzione del prodotto software.

Nel contesto del progetto *Second Brain_G*, e in accordo con il tailoring descritto nella Sezione 2, il gruppo *Synergo Unit* adotta i seguenti processi primari:

- **Fornitura:** disciplina il rapporto tra il gruppo, il committente e la proponente, regolando candidatura, pianificazione, produzione degli artefatti, consegna e comunicazioni ufficiali;
- **Sviluppo:** disciplina le attività tecniche necessarie alla definizione e alla progressiva realizzazione del prodotto software, con particolare riferimento, in fase RTB, all'analisi dei requisiti, all'analisi dello stato dell'arte, alla progettazione preliminare e all'implementazione dimostrativa del Proof of Concept.

Le attività di codifica estesa, testing automatico, integrazione continua e rilascio software verranno progressivamente normate nella fase PB, in seguito alla stabilizzazione dell'architettura e all'avvio dello sviluppo del prodotto.

3.1 Processo di Fornitura

3.1.1 Descrizione

Il processo di fornitura regola le attività mediante cui il gruppo *Synergo Unit* opera come fornitore nei confronti del committente e della proponente.

Nel contesto del progetto didattico:

- il **docente** ricopre il ruolo_G di committente rispetto alle forniture, alle scadenze, ai vincoli regolamentari e alla valutazione delle baseline;
- la **proponente**, Zuccheti S.p.A., ricopre il ruolo di cliente rispetto alle esigenze di prodotto e di mentore rispetto alle scelte tecniche e progettuali;
- il gruppo *Synergo Unit* ricopre il ruolo di fornitore, assumendosi la responsabilità di produrre, verificare e consegnare gli artefatti richiesti dalle revisioni di avanzamento.

Il processo comprende:

- la valutazione dei capitolati_G;
- la candidatura al capitolato scelto;
- la formalizzazione degli impegni economici e temporali;
- la produzione della documentazione richiesta per ciascuna baseline;
- la gestione delle comunicazioni ufficiali;
- la rendicontazione dell'avanzamento;
- la consegna della documentazione tramite repository_G pubblico e sito web.

3.1.2 Norme e Strumenti del Processo di Fornitura

3.1.2.1 Strumenti a Supporto Il processo di fornitura è supportato dai seguenti strumenti:

- **Email istituzionale del gruppo:** l'indirizzo `synergounit.20@gmail.com` viene utilizzato per le comunicazioni ufficiali con committente e proponente, garantendo tracciabilità e continuità nelle interazioni esterne;
- **Repository GitHub:** utilizzato come punto di raccolta, versionamento e pubblicazione degli artefatti documentali e del codice del Proof of Concept;
- **GitHub Pages_G:** utilizzato per rendere consultabile la documentazione prodotta;
- **GitHub Projects:** utilizzato per il tracciamento delle attività, delle milestone_G RTB e PB e dello stato di avanzamento;
- **Google Sheets:** utilizzato per la raccolta e la consuntivazione delle ore previste ed effettive;
- **Google Forms:** utilizzato per la creazione standardizzata delle attività e il successivo tracciamento su GitHub;
- **Canva:** utilizzato per la predisposizione del materiale di supporto ai Diari di Bordo.

Le modalità operative di utilizzo degli strumenti sono dettagliate nei processi di supporto e nei processi organizzativi.

3.1.2.2 Documentazione prodotta in fase CC Durante la fase di candidatura (CC), il gruppo ha prodotto e approvato la documentazione necessaria alla presentazione della propria proposta per il capitolato C6 – *Second Brain*.

Documento	Descrizione
<u>Lettera di Presentazione_G</u>	Documento con cui il gruppo comunica formalmente la candidatura al capitolato scelto. Contiene l'indicazione del capitolato, della proponente, della scadenza prevista, del budget _G preventivato e degli impegni assunti dal gruppo.
<u>Valutazione dei Capitolati</u>	Documento esterno che raccoglie il confronto tra i capitolati analizzati, evidenziando per ciascuno aspetti positivi, criticità, rischi e motivazioni della scelta finale.
<u>Preventivo Costi e Assunzione Impegni</u>	Documento esterno che formalizza il preventivo economico, la distribuzione delle ore produttive, la ripartizione del lavoro per ruolo e gli impegni assunti dal gruppo verso il committente.
Norme di Progetto	Documento interno, approvato in versione 1.x.x per la baseline CC, contenente le norme operative iniziali del gruppo.

Continua nella pagina successiva

Documento	Descrizione
Glossario	Documento interno, approvato in versione 1.x.x per la baseline CC, contenente i termini tecnici e ambigui utilizzati nella documentazione iniziale.
Verbalì interni	Documenti interni che tracciano decisioni organizzative, assegnazioni di attività e discussioni svolte dal gruppo.
Verbalì esterni	Documenti esterni che formalizzano gli incontri con le aziende proponenti e i chiarimenti ottenuti durante la fase di candidatura.

La versione 1.x.x delle Norme di Progetto e del Glossario è relativa alla sola fase di candidatura. Con l'ingresso nella fase RTB, tali documenti vengono aggiornati alla versione 2.x.x per recepire i nuovi processi attivati.

3.1.2.3 Documentazione prodotta o prevista in fase RTB Durante la fase RTB, il gruppo produce e aggiorna la documentazione necessaria alla Requirements & Technology Baseline_G.

Documento	Redattore _G	Uso	Descrizione
Lettera di Presentazione RTB	Responsabile _G	Esterno	Documento redatto prima della candidatura alla RTB. Formalizza la richiesta di partecipazione alla revisione _G e rimanda alla documentazione disponibile nel repository pubblico.
<u>Piano di Progetto</u>	Amministratore _G	Esterno	Documento che descrive pianificazione, consuntivazione, gestione dei rischi, rotazione dei ruoli, avanzamento degli sprint _G e controllo dei costi.
<u>Piano di Qualifica</u>	Amministratore	Esterno	Documento che definisce strategie, metriche e attività di verifica e validazione adottate dal gruppo per garantire la qualità di processo e di prodotto.
<u>Analisi dei Requisiti</u>	Analista _G	Esterno	Documento tecnico che formalizza casi d'uso _G , requisiti, fonti e tracciabilità tra esigenze espresse, funzionalità attese e vincoli del prodotto.

Continua nella pagina successiva

Documento	Redattore	Uso	Descrizione
Norme di Progetto	Amministratore	Interno	Documento che definisce il way of working del gruppo. In RTB viene aggiornato alla versione 2.x.x per includere i processi attivi dopo la candidatura.
<u>Glossario</u>	Amministratore	Interno	Documento contenente termini tecnici, acronimi ed espressioni ambigue. Ogni autore propone i termini emersi durante la stesura; l'Amministratore ne cura normalizzazione e coerenza.
<u>Analisi dello Stato dell'Arte</u>	Analista / Progettista _G	Interno	Documento che raccoglie la valutazione comparativa delle tecnologie e la scelta dello stack candidato. L'Analista cura la valutazione; il Progettista seleziona le tecnologie sulla base delle esigenze architetturali.
Verbalì interni	Responsabile	Interno	Documenti che tracciano riunioni, decisioni operative, criticità, retrospettive e pianificazione interna.
Verbalì esterni	Amministratore	Esterno	Documenti che formalizzano incontri con la proponente e chiarimenti rilevanti ai fini dell'Analisi dei Requisiti e delle scelte tecnologiche.
Diari di Bordo	Responsabile	Esterno	Slide e materiale gestionale utilizzati per il confronto periodico con il committente, con evidenza di avanzamento, criticità e obiettivi successivi.

I destinatari dei documenti esterni sono:

- il gruppo *Synergo Unit*;
- Zucchetti S.p.A.;
- il Prof. Tullio Vardanega;
- il Prof. Riccardo Cardin.

I Diari di Bordo devono riportare almeno:

- risultati raggiunti nel periodo di riferimento;
- obiettivi previsti per il periodo successivo;
- criticità emerse;

- eventuali scostamenti rispetto alla pianificazione;
- decisioni o richieste di chiarimento da sottoporre al committente.

Tale materiale ha funzione di supporto alla comunicazione periodica con il committente e non sostituisce la documentazione ufficiale di progetto.

Il Proof of Concept non prevede, nella fase RTB, un documento separato dedicato: le sue finalità, il perimetro e gli esiti verranno descritti all'interno del materiale di presentazione della Requirements & Technology Baseline e, ove necessario, nei Diari di Bordo.

Gli Architecture Decision Records (ADR) non sono ancora considerati artefatti attivi della fase RTB; la loro introduzione è pianificata per la fase PB, in seguito alla validazione preliminare delle scelte tecnologiche e architetturali.

3.1.3 Attività del Processo di Fornitura

3.1.3.1 Inizializzazione

- **Ruoli:** Responsabile, Amministratore.
- **Input:** capitolati disponibili, regolamento del progetto didattico, indicazioni dei docenti.
- **Output:** decisione di partecipazione alla selezione del capitolato.

L'attività consiste nell'analisi preliminare delle proposte disponibili, nella valutazione della compatibilità tra competenze del gruppo e richieste del capitolato e nella decisione di procedere con la candidatura.

3.1.3.2 Risposta

- **Ruoli:** Responsabile.
- **Input:** capitolato selezionato, valutazione dei capitolati, preventivo.
- **Output:** Lettera di Presentazione.

Il gruppo formalizza la propria candidatura mediante una Lettera di Presentazione, nella quale dichiara il capitolato scelto, la motivazione della scelta, la scadenza prevista, il budget preventivato e gli impegni assunti.

3.1.3.3 Consolidamento degli impegni

- **Ruoli:** Responsabile, Amministratore.
- **Input:** candidatura accettata, Preventivo Costi e Assunzione Impegni.
- **Output:** impegni economici e temporali consolidati.

A seguito dell'aggiudicazione, il gruppo considera vincolanti gli impegni dichiarati in fase di candidatura.

Gli impegni assunti prevedono:

- completamento del progetto entro il 2026-08-28;

- rispetto del budget preventivato pari a 11.665,00 €;
- mantenimento del monte ore produttivo dichiarato;
- rotazione dei ruoli secondo quanto pianificato nel Piano di Progetto;
- comunicazione costante e tracciabile con committente e proponente.

Il costo preventivato costituisce il limite massimo previsto per l'intero svolgimento del progetto.

3.1.3.4 Pianificazione

- **Ruoli:** Amministratore, Responsabile.
- **Input:** impegni approvati, milestone di progetto, baseline prevista.
- **Output:** Piano di Progetto aggiornato.

L'Amministratore redige e aggiorna il Piano di Progetto, includendo pianificazione, preventivi, consuntivi di sprint, rischi, rotazione dei ruoli e avanzamento rispetto alle milestone.

3.1.3.5 Esecuzione e controllo

- **Ruoli:** tutti i ruoli attivi nello sprint.
- **Input:** Piano di Progetto, attività pianificate, issue_G assegnate.
- **Output:** documentazione prodotta, consuntivi aggiornati, avanzamento monitorato.

Durante ogni sprint il gruppo svolge le attività pianificate, consuntiva le ore effettive e aggiorna lo stato delle issue. Il controllo dell'avanzamento avviene tramite GitHub Projects e fogli di consuntivazione condivisi.

3.1.3.6 Revisione

- **Ruoli:** Responsabile, Verificatore_G, ruoli coinvolti negli artefatti.
- **Input:** documenti o materiali prodotti, feedback interno o esterno.
- **Output:** artefatti corretti, feedback integrato, eventuali nuove issue.

Il gruppo revisiona periodicamente gli artefatti prodotti tramite verifiche interne, retrospettive di sprint, incontri con la proponente e Diari di Bordo con il committente. Eventuali osservazioni vengono trasformate in attività tracciate.

3.1.3.7 Validazione esterna

- **Ruoli:** Responsabile, Analista, Progettista.
- **Input:** requisiti, dubbi interpretativi, scelte tecnologiche preliminari, materiale di supporto.
- **Output:** chiarimenti formalizzati, verbale esterno approvato, eventuali modifiche all'Analisi dei Requisiti, all'Analisi dello Stato dell'Arte o al perimetro del Proof of Concept.

La validazione esterna ha lo scopo di verificare con la proponente la correttezza delle interpretazioni assunte dal gruppo, in particolare per gli aspetti riguardanti il perimetro funzionale, l'interazione con LLM, la gestione delle note Markdown, il distant writing_G e i requisiti prestazionali.

Ogni chiarimento rilevante deve essere riportato in un verbale esterno e, se necessario, tradotto in modifiche tracciate alla documentazione.

I verbali esterni devono essere condivisi con la proponente per confermare la correttezza dei contenuti riportati e degli accordi emersi.

3.1.3.8 Consegna e completamento

- **Ruoli:** Responsabile, Amministratore.
- **Input:** documentazione verificata e approvata.
- **Output:** baseline pubblicata e candidatura alla revisione.

La consegna della documentazione avviene tramite pubblicazione nel repository pubblico e nel sito web associato. La candidatura alla revisione viene effettuata tramite email, riportando il puntatore alla documentazione aggiornata, senza invio di allegati salvo diversa indicazione del committente.

3.2 Processo di Sviluppo

3.2.1 Descrizione

Il processo di sviluppo disciplina le attività tecniche che portano alla definizione e alla progressiva realizzazione del prodotto software.

Il processo di sviluppo adottato dal gruppo è istanziato a partire dal processo *Development* definito dallo standard ISO/IEC 12207.

Esso comprende le attività riportate di seguito, che vengono attivate progressivamente durante il ciclo di vita del progetto in funzione dello stato di avanzamento della fornitura.

- Implementazione del processo (Process implementation);
- Analisi dei requisiti di sistema (System requirements analysis);
- Progettazione dell'architettura di sistema (System architectural design);
- Analisi dei requisiti software (Software requirements analysis);
- Progettazione dell'architettura software (Software architectural design);
- Progettazione dettagliata del software (Software detailed design);
- Codifica e test del software (Software coding and testing);
- Integrazione del software (Software integration);
- Test di qualifica del software (Software qualification testing);
- Integrazione del sistema (System integration);

- Test di qualifica del sistema (System qualification testing);
- Installazione del software (Software installation);
- Supporto all'accettazione del software (Software acceptance support).

Con l'avvio dello Sprint 9, a partire dal 2026-06-02, il processo di sviluppo include attività di codifica finalizzate alla realizzazione del Proof of Concept.

Tali attività hanno lo scopo di validare tecnicamente le scelte effettuate, dimostrare la fattibilità dei principali flussi funzionali individuati per la Requirements & Technology Baseline e fornire evidenze utili alla successiva progettazione di dettaglio.

3.2.2 Implementazione del processo

Il gruppo adotta un modello organizzativo incrementale ispirato a Scrum, basato su sprint settimanali con cadenza da martedì a martedì.

Il workflow prevede:

- sprint review_G dello sprint precedente;
- sprint retrospective_G dello sprint precedente;
- sprint planning_G dello sprint corrente;
- daily stand-up_G asincrono;
- riunione intermedia di allineamento;
- monitoraggio continuo delle attività.

La pianificazione dettagliata degli sprint, la rotazione dei ruoli e le milestone sono riportate nel Piano di Progetto.

Il gruppo adotta un approccio incrementale alla progettazione e allo sviluppo.

Le decisioni tecniche e architetturali vengono raffinate progressivamente sulla base di:

- feedback della proponente;
- risultati del Proof of Concept;
- criticità emerse durante gli sprint;
- evoluzione dei requisiti;
- vincoli rilevati durante l'analisi dello stato dell'arte.

Tale approccio consente di evitare scelte premature e di mantenere coerenza tra requisiti, tecnologie e architettura.

3.2.3 Analisi dei Requisiti di sistema

3.2.3.1 Scopo L'analisi dei requisiti di sistema ha lo scopo di identificare, classificare, tracciare e mantenere aggiornati i requisiti del progetto Second Brain.

3.2.3.2 Raccolta dei requisiti I requisiti vengono raccolti a partire da:

- capitolato d'appalto C6 – *Second Brain*;
- incontri con la proponente (verbali esterni);
- analisi del dominio applicativo.

3.2.3.3 Classificazione I requisiti vengono classificati secondo:

- tipologia;
- importanza;
- fonte;
- casi d'uso associati.

3.2.3.4 Gestione delle modifiche Le modifiche ai requisiti devono:

- essere discusse internamente;
- essere tracciate tramite issue;
- mantenere consistenti le matrici di tracciabilità;
- essere riportate nei verbali quando derivano da incontri esterni;
- essere validate dalla proponente quando incidono sul perimetro funzionale o sui vincoli del prodotto.

3.2.4 Progettazione dell'architettura di sistema

3.2.4.1 Scopo La progettazione dell'architettura di sistema ha lo scopo di identificare le componenti hardware, software e le operazioni manuali

Il nostro progetto è prettamente software e non identifica componenti hardware.

3.2.5 Analisi dei requisiti software

3.2.5.1 Scopo L'analisi dei requisiti software ha lo scopo di stabilire e documentare le specifiche tecniche, funzionali, di qualità e di interfaccia necessarie alla progettazione e alla realizzazione di ogni componente software del sistema

3.2.5.2 Nomenclatura dei requisiti Ogni requisito deve essere identificato tramite un codice univoco nel formato:

R[ID] - [Tipo] - [Importanza]

dove:

- **ID**: identificativo numerico progressivo;

- **Tipo:** indica la categoria del requisito:
 - F: funzionale;
 - Q: qualità;
 - V: vincolo;
 - P: prestazionale;
- **Importanza:** indica la priorità del requisito:
 - O: obbligatorio;
 - D: desiderabile;
 - Op: opzionale.

La nomenclatura deve essere mantenuta stabile per tutta la durata della baseline. Eventuali modifiche agli identificativi devono essere motivate, tracciate e verificate per non compromettere la consistenza delle matrici di tracciabilità.

3.2.5.3 Tracciabilità Il gruppo mantiene matrici di tracciabilità tra:

- fonti e requisiti;
- requisiti e casi d'uso;
- casi d'uso e requisiti.

La tracciabilità deve garantire:

- copertura completa delle fonti;
- assenza di requisiti privi di motivazione;
- coerenza tra requisiti e casi d'uso;
- possibilità di risalire da ogni requisito alla fonte che lo giustifica;
- possibilità di risalire da ogni caso d'uso ai requisiti soddisfatti;
- supporto alla successiva attività di verifica e validazione.

3.2.5.4 Gestione dei Casi d'Uso I casi d'uso costituiscono il principale strumento di descrizione delle interazioni tra utente e sistema nella fase RTB.

Essi sono organizzati nelle seguenti macroaree funzionali:

- **Gestione editor e Markdown:** scrittura, formattazione_G e rendering;
- **Interazione AI e LLM:** generazione di suggerimenti, riscrittura, distant writing, applicazione di stili e supporto tramite tecniche come i *Sei Cappelli Per Pensare_G*;
- **Gestione I/O e persistenza:** apertura, salvataggio, importazione ed esportazione delle note.

3.2.5.5 Convenzioni dei casi d'uso

I casi d'uso adottano le seguenti convenzioni:

- **Denominazione:** ogni caso d'uso deve seguire il formato **UC X: Titolo**, dove **X** rappresenta l'identificativo numerico progressivo e **Titolo** descrive sinteticamente l'interazione modellata;
- **Sottocasi:** eventuali casi d'uso derivati devono mantenere una numerazione gerarchica coerente con il caso d'uso principale;
- **Ordinamento:** i diagrammi vengono mantenuti in ordine numerico e aggiornati in modo incrementale;
- **Strumenti:** i diagrammi UML_G vengono realizzati e mantenuti tramite LucidChart/LucidSpark_G o strumenti equivalenti condivisi dal gruppo;
- **Navigabilità:** i riferimenti interni tra casi d'uso devono essere realizzati tramite collegamenti interni al documento;
- **Inclusioni ed estensioni:** le relazioni tra casi d'uso devono essere esplicitate sia nei diagrammi sia nella descrizione testuale.

3.2.6 Progettazione dell'architettura software

3.2.6.1 Scopo La progettazione dell'architettura software ha lo scopo di individuare una prima struttura del sistema, coerente con i requisiti e con le tecnologie selezionate. Tale attività trasforma i requisiti software in un'architettura che ne descrive la struttura top-level e identifica i componenti software necessari.

In fase RTB, tale attività ha carattere esplorativo e mira a:

- individuare i principali moduli del sistema;
- valutare le interazioni tra frontend, backend e servizi AI;
- giustificare la selezione dello stack tecnologico sulla base dell'Analisi dello Stato dell'Arte;
- evidenziare rischi tecnici;
- preparare il perimetro del Proof of Concept;
- raccogliere elementi utili per la progettazione di dettaglio in PB.

3.2.6.2 Integrazione dello Stato dell'Arte e selezione tecnologica La definizione dell'architettura si fonda sui risultati dell'Analisi dello Stato dell'Arte, che funge da motore decisionale per la scelta dei componenti. L'architettura viene valutata secondo criteri di appropriatezza dei metodi e degli standard di progettazione.

Il processo di selezione prevede la collaborazione tra:

- **Analista:** cura la valutazione comparativa delle alternative tecnologiche basandosi su maturità, supporto della comunità e compatibilità con il capitolato;
- **Progettista:** seleziona le tecnologie (attualmente individuate in Vue.js, FastAPI e Docker) che meglio rispondono alle esigenze di modularità e integrazione dei servizi AI individuate nel design.

Le decisioni architetture definitive verranno consolidate nella fase PB. Gli ADR verranno introdotti quando il gruppo avrà stabilizzato il processo decisionale architetture e saranno utilizzati per documentare in modo tracciabile le scelte rilevanti, le alternative valutate e le motivazioni della decisione finale.

3.2.6.3 Stile Architetture (Fase PB) A seguito del consolidamento delle decisioni nella fase PB, il sistema adotta un'architettura ibrida a due stili combinati, separati da un confine REST:

- **Frontend:** adotta il pattern MVC (Model-View-Controller) con Observer in modalità *pull*. Questa scelta risolve la coordinazione tra editor, anteprima e proposte AI senza accoppiare le View tra loro.
- **Backend:** adotta una *Layered Architecture* (Presentation / Business / Persistence). Tale scelta garantisce l'indipendenza del dominio (AI, orchestrazione note) rispetto al provider LLM e al tipo di storage, limitando la complessità rispetto a un'architettura esagonale, come concordato con la proponente.

3.2.7 Progettazione dettagliata del software

3.2.7.1 Scopo La progettazione dettagliata del software definisce il design di ogni unità software in modo che possa essere codificata. Nella fase PB, questa attività stabilisce i pattern implementativi e la gestione delle interazioni tra componenti.

3.2.7.2 Gestione delle Dipendenze (Dependency Injection) Per rispettare il principio *Composition over Inheritance*, nessuna classe applicativa crea le proprie dipendenze direttamente.

- **Backend:** l'iniezione delle dipendenze avviene tramite la funzione `Depends()` di FastAPI, con provider centralizzati nel package `app/di/`.
- **Frontend:** in assenza di un container DI nativo in Vue, l'iniezione è manuale e centralizzata nel modulo `src/bootstrap/composition.ts`, mantenendo il dominio TypeScript puro e isolato dal framework.

3.2.8 Codifica e test del software

3.2.8.1 Scopo La codifica e test del software riguarda lo sviluppo delle unità software, dei database e delle relative procedure di test

Con l'avvio dello Sprint 9, a partire dal 2026-06-02, il gruppo avvia l'implementazione dimostrativa del Proof of Concept.

Il Proof of Concept ha lo scopo di dimostrare la fattibilità tecnica delle scelte effettuate e la possibilità di soddisfare i requisiti principali individuati per la Requirements & Technology Baseline.

Il PoC_G costituisce una baseline tecnologica e non architetture: esso ha valore dimostrativo e può essere rivisto nella successiva fase PB, qualora le evidenze raccolte lo rendano opportuno.

La codifica svolta in questa fase non costituisce ancora il processo completo di sviluppo software previsto per la fase PB.

3.2.8.2 Perimetro del PoC Il perimetro minimo del PoC comprende:

- editor Markdown basato su CodeMirror;
- visualizzazione split tra editor e anteprima;
- rendering dell'anteprima Markdown;
- applicazione della formattazione in grassetto;
- salvataggio locale di contenuti Markdown;
- importazione di contenuti Markdown;
- apertura dell'editor su una nota esistente;
- interfaccia per l'interazione con LLM;
- popup di proposta generata dall'LLM;
- azioni di accettazione e rifiuto della proposta;
- selezione del cappello da applicare al testo selezionato;
- interfaccia di Distant Writing con inserimento del prompt_G.

Il PoC non prevede, in fase RTB, persistenza su database. La gestione dei contenuti è limitata a dati locali o file Markdown.

3.2.8.3 Tecnologie del PoC Il PoC utilizza:

- Vue.js per il frontend;
- CodeMirror per l'editor Markdown;
- FastAPI per il backend;
- Docker Compose per l'esecuzione coordinata dei servizi;
- due container: **frontend** e **backend**.

La struttura del PoC è collocata nella cartella **poc/** della repository **Second-Brain**, con separazione tra frontend e backend.

3.2.8.4 Criteri di successo Il PoC è considerato soddisfacente se consente di:

- avviare localmente frontend e backend tramite Docker Compose;
- dimostrare la comunicazione minima tra frontend e backend;
- visualizzare un editor Markdown funzionante;
- mostrare l'anteprima del contenuto Markdown tramite visualizzazione split;
- applicare almeno una formattazione al testo;
- mostrare l'interfaccia prevista per l'interazione con LLM;

- mostrare il flusso di accettazione o rifiuto di una proposta generata;
- mostrare l'interfaccia di Distant Writing con inserimento del prompt;
- raccogliere evidenze utili alla progettazione di dettaglio in PB;
- individuare eventuali rischi tecnici da mitigare nella fase PB.

3.2.8.5 Validazione del PoC Il PoC viene validato internamente dal Progettista e dal Verificatore.

Il confronto con la proponente avviene prima della RTB ed è gestito dal Responsabile, con il supporto dei membri che hanno lavorato sugli aspetti tecnici e funzionali coinvolti.

I risultati del Proof of Concept devono:

- essere discussi internamente;
- essere confrontati con la proponente;
- essere presentati durante la Requirements & Technology Baseline;
- guidare la successiva progettazione di dettaglio in PB.

Eventuali osservazioni della proponente devono essere riportate in verbale esterno e, se producono modifiche operative, trasformate in issue.

3.2.8.6 Sviluppo del Prodotto (Fase PB) Con l'ingresso nella fase PB, l'attività di codifica si estende allo sviluppo del prodotto finale (Minimum Viable Product). Per garantire uniformità e coerenza nello sviluppo collaborativo, il gruppo ha formalizzato regole ferree per la stesura del codice.

3.2.8.7 Convenzioni di Nomenclatura (Naming) Tutti i membri, nel ruolo di Programmatore, devono attenersi rigorosamente alle seguenti convenzioni:

- **Classi (TS e Python):** PascalCase.
- **Interfacce/Servizi (TS):** Nome del ruolo, senza prefisso "I" o suffissi tecnici (es. `NoteService`, non `INoteService`).
- **Adapter e Repository:** Suffisso `Adapter` o `Repository` in base al pattern (es. `OpenAIAdapter`, `NoteRepository`). Un'eccezione è il pattern `Proxy` per il frontend (es. `NoteServiceProxy`).
- **Metodi e Variabili (TypeScript):** camelCase.
- **Moduli e Funzioni (Python):** snake_case in accordo allo standard PEP8.
- **File di Test:** Devono avere lo stesso nome della classe testata, con suffisso `.test.ts` per il frontend e prefisso `test_*.py` per il backend.

3.2.9 Integrazione del software

3.2.9.1 Scopo Integrazione delle unità software nel "software item" completo e relativi test.

3.2.9.2 Workflow Git e CI/CD L'integrazione delle unità software avviene attraverso il modello di trunk-based development, nel quale le modifiche apportate ai vari moduli confluiscono nel branch stabile main esclusivamente tramite Pull Request verificate. A partire dagli Sprint 12 e 13, viene avviato il setup dell'ambiente di Continuous Integration/Continuous Deployment (CI/CD) per automatizzare i processi di build e verifica.

3.2.9.3 Test di Integrazione (TI) L'efficacia dell'unione dei moduli è validata tramite appositi Test di Integrazione (TI) volti a controllare la comunicazione tra le unità (es. frontend e backend), la cui pianificazione di dettaglio è prevista per la fase di Product Baseline (PB).

3.2.10 Test di qualifica del software

3.2.10.1 Scopo Il test di qualifica del software verifica che il prodotto software soddisfi i propri requisiti.

3.2.10.2 Mappatura UC-Test La verifica dei requisiti software si basa sulla Mappatura Test di Sistema - Casi d'Uso, che definisce un collegamento univoco tra ogni funzionalità individuata nell'Analisi dei Requisiti e uno specifico test di sistema (TS).

3.2.10.3 Metriche di Prodotto Il superamento di tali test deve essere accompagnato dal monitoraggio di metriche oggettive, tra cui il Soddisfacimento dei Requisiti Obbligatori (MPD01), il cui valore deve attestarsi tassativamente al 100% per garantire la piena conformità del prodotto alle specifiche concordate con la proponente.

3.2.11 Integrazione del sistema

3.2.11.1 Scopo Integrazione del software con hardware e altre componenti di sistema.

3.2.11.2 Interazione con attori esterni Sebbene il progetto sia prettamente software, l'integrazione di sistema riguarda la connessione sinergica tra Frontend (Vue.js), Backend (FastAPI) e i servizi LLM esterni (Gemma, ChatGPT-OSS o Qwen) tramite API aderenti allo standard OpenAI.

3.2.11.3 Containerizzazione Tale processo è facilitato e validato attraverso l'uso di Docker Compose, che permette l'esecuzione coordinata dei diversi servizi in ambienti containerizzati isolati e riproducibili.

3.2.12 Test di qualifica del sistema

3.2.12.1 Scopo Il test di qualifica del sistema è il test finale per dimostrare che il sistema completo è pronto per la consegna.

3.2.12.2 Validazione della Technology Baseline I test di qualifica del sistema in fase RTB sono finalizzati alla validazione della Technology Baseline. Essi dimostrano che l'intera infrastruttura (interfaccia utente, logica di backend e interazione AI) funzioni correttamente secondo gli scenari d'uso principali descritti nell'Analisi dei Requisiti.

3.2.13 Installazione del software

3.2.13.1 Scopo L'installazione del software è la pianificazione ed esecuzione dell'installazione nell'ambiente target.

3.2.13.2 Manuale Tecnico (README) La procedura di installazione e configurazione nell'ambiente target è documentata nel file README presente nella cartella poc/ del repository. Tale documento riporta i prerequisiti necessari, la procedura di avvio tramite Docker Compose e le modalità di verifica del corretto funzionamento dell'intero sistema su host locale.

3.2.14 Supporto all'accettazione del software

3.2.14.1 Scopo Il supporto all'accettazione del software riguarda l'attività di supporto al cliente durante le revisioni e i test di accettazione finale.

3.2.14.2 Validazione con la Proponente Il processo prevede inoltre incontri di validazione periodici con la proponente Zucchetti, i cui esiti (accordi e correzioni richieste) sono formalizzati in appositi verbali esterni per guidare l'approvazione finale del prodotto.

4 Processi di Supporto

In conformità allo standard ISO/IEC 12207:1995, i processi di supporto comprendono le attività trasversali necessarie a garantire qualità, tracciabilità, coerenza e controllo degli artefatti prodotti.

Nel contesto del progetto *Second Brain*, e in accordo con il tailoring descritto nella Sezione 2, il gruppo *Synergo Unit* adotta i seguenti processi di supporto:

- documentazione;
- gestione della configurazione;
- verifica;
- validazione;
- accertamento della qualità;
- gestione dei cambiamenti;
- gestione delle non conformità;

4.1 Processo di Documentazione

4.1.1 Descrizione

Il processo di documentazione definisce regole, strumenti e ciclo di vita per la produzione, la revisione, l'approvazione e la pubblicazione della documentazione del gruppo.

Esso si applica a ogni documento gestionale o tecnico, interno o esterno, prodotto dal gruppo *Synergo Unit*. Il Diario di Bordo_G segue un template_G separato realizzato tramite Canva e non prevede registro delle modifiche.

La documentazione deve essere:

- uniforme nella struttura;
- coerente con le presenti Norme di Progetto;
- verificabile tramite Pull Request_G;
- tracciabile rispetto alle modifiche logiche effettuate;
- consultabile tramite repository e sito documentale.

4.1.2 Tipologia dei Documenti

4.1.2.1 Documentazione Gestionale Interna

- **Norme di Progetto:** definiscono processi, convenzioni, strumenti e procedure operative adottate dal gruppo. Sono redatte e mantenute dall'Amministratore.
- **Glossario:** raccoglie termini tecnici, acronimi ed espressioni ambigue utilizzate nella documentazione. Ogni autore può proporre nuovi termini; l'Amministratore ne cura normalizzazione e coerenza.

- **Analisi dello Stato dell'Arte:** documento che raccoglie valutazione comparativa delle tecnologie e motivazione dello stack candidato per il Proof of Concept.
- **Verbali Interni:** documentano riunioni interne, decisioni operative, retrospettive, pianificazione e assegnazione dei task_G.

4.1.2.2 Documentazione Gestionale Esterna

- **Verbali Esterni:** documentano gli incontri con la proponente e con gli stakeholder_G esterni. Sono redatti dall'Amministratore e condivisi con la proponente per conferma e firma.
- **Piano di Progetto:** descrive pianificazione, consuntivazione, gestione dei rischi, controllo dei costi e rotazione dei ruoli.
- **Piano di Qualifica:** definisce strategie, metriche e attività di verifica e validazione adottate dal gruppo.
- **Diario di Bordo:** materiale gestionale usato per il confronto periodico con il committente ed è redatto tramite template Canva.

Il template del Diario di Bordo deve garantire l'identificazione del materiale presentato, riportando in modo chiaro almeno il gruppo, il riferimento al corso e la data del Diario di Bordo.

Ogni pagina, ad eccezione del frontespizio, deve riportare un footer contenente almeno:

- numero progressivo del Diario di Bordo;
- data nel formato AAAA/MM/GG;
- numero di pagina e numero totale di pagine;
- nome del gruppo;
- numero del gruppo.

La struttura prevista per il footer è:

Diario di Bordo N° - AAAA/MM/GG
Synergo Unit - Gruppo 20
numero pagina / numero pagine totali

- **Sito Web:** punto di accesso alla documentazione pubblicata tramite GitHub Pages.

4.1.2.3 Documentazione Tecnica

- **Analisi dei Requisiti:** documento tecnico esterno che formalizza attori, casi d'uso, requisiti, fonti e matrici di tracciabilità.

4.1.3 Strumenti di Documentazione

La documentazione viene prodotta tramite:

- **L^AT_EX con Visual Studio Code:** ambiente principale di redazione;

- **LaTeX Workshop:** estensione utilizzata per compilare localmente i documenti PDF;
- **Repository GitHub:** archivio ufficiale della documentazione e della relativa storia;
- **GitHub Pages:** sito documentale aggiornato periodicamente tramite task dedicato;
- **Canva:** strumento utilizzato per il template dei Diari di Bordo;
- **Script Python per Glossario:** strumento utilizzato per applicare automaticamente il pedice_G alla prima occorrenza dei termini presenti nel Glossario.

Ogni membro lavora esclusivamente su copia locale del repository.

4.1.4 Convenzioni Redazionali

Al fine di garantire uniformità e coerenza tra i documenti, il gruppo adotta le seguenti convenzioni:

- **Nomi dei file:** tutti i nomi devono essere scritti in minuscolo, utilizzando il carattere `_` come separatore di parola.
Esempio: `piano_di_progetto.tex`.
- **Nome del file principale:** il file principale `LaTeX` deve essere nominato secondo il nome previsto per il PDF finale.
Esempio: `norme_di_progetto.tex` produce `norme_di_progetto.pdf`.
- **Titoli:** sezioni, sottosezioni e paragrafi devono utilizzare Title Case_G italianizzato, con maiuscola sulle parole semanticamente rilevanti e minuscola su articoli, preposizioni e congiunzioni interne.
- **Tabelle lunghe:** le tabelle che possono superare una pagina devono essere realizzate tramite `xltabular` o `longtable`, in modo da consentirne la spezzatura automatica.
- **Pedice_G:** la prima occorrenza, nell'intero documento, di ogni termine presente nel Glossario deve essere marcata con il pedice_G. Lo script riceve in ingresso la concatenazione dei file `LaTeX` delle sezioni e applica il pedice alla prima occorrenza globale.
- **Link generici:** i collegamenti ipertestuali nel testo devono essere prodotti tramite il comando `\link{url}{testo}`.
- **Link documentali in tabelle:** i collegamenti a documenti all'interno di tabelle devono essere prodotti tramite il comando `\doclink{url}{testo}`, così da preservare leggibilità e andata a capo.
- **Issue GitHub:** i riferimenti a issue devono essere prodotti tramite il comando `\ghissue{id}`.
- **Decisioni da verbale interno:** le decisioni interne devono essere identificate con codice `VI-y.xG`, dove `y` è il numero progressivo del verbale e `x` è il numero progressivo della decisione.
- **Decisioni da verbale esterno:** le decisioni esterne devono essere identificate con codice `VE-y.xG`, con la stessa logica adottata per i verbali interni.

4.1.5 Template Documentale

Tutti i documenti, ad eccezione dei Diari di Bordo, condividono un template $\text{\LaTeX}$ comune comprensivo di:

- frontespizio;
- registro delle modifiche;
- indice;
- sezioni specifiche del documento;
- configurazioni condivise;
- loghi e asset comuni.

Il template definisce inoltre:

- margini;
- colori;
- profondità dell'indice;
- comandi personalizzati;
- configurazione dei link;
- formattazione delle tabelle;
- stile dei paragrafi.

4.1.6 Registro delle Modifiche

Ogni documento $\text{\LaTeX}$, ad eccezione dei Diari di Bordo, deve contenere un registro delle modifiche.

Il registro deve riportare:

- **Versione;**
- **Data;**
- **Autore;**
- **Verificatore;**
- **Descrizione.**

Il registro delle modifiche è aggiornato dal Redattore.

Le voci del registro devono descrivere modifiche logiche e documentali significative.

4.1.7 Gestione del Glossario

Il Glossario viene aggiornato in modo incrementale durante la produzione della documentazione.

Il processo prevede:

1. durante la stesura, ogni autore individua eventuali nuovi termini tecnici, acronimi o espressioni ambigue;
2. al termine della stesura, l'autore inserisce i nuovi termini nel file `termini.tex` del Glossario;
3. l'autore aggiorna il file `glossary.txt`, utilizzato dallo script Python;
4. l'autore esegue manualmente lo script per applicare il pedice_G alla prima occorrenza dei termini nel documento;
5. il Verificatore controlla che i termini inseriti siano coerenti con il documento e che il pedice_G sia stato applicato correttamente.

L'Amministratore può intervenire per normalizzare definizioni, duplicati, maiuscole/minuscole e sinonimi.

4.1.8 Checklist Prima della Pull Request

Prima di aprire una Pull Request, il Redattore deve verificare che:

- il documento compili senza errori bloccanti;
- il PDF sia generato correttamente;
- il registro delle modifiche sia aggiornato, così come numero di versione e data di ultima modifica;
- non siano presenti segnaposto non motivati;
- i link siano funzionanti;
- i riferimenti interni siano corretti;
- le tabelle lunghe siano spezzabili;
- il Glossario sia aggiornato, se necessario;
- il pedice_G sia stato applicato correttamente;
- il nome del file sia conforme alle convenzioni.

4.1.9 Ciclo di Vita Documentale_G

Ogni documento attraversa i seguenti stati:

1. **In Stesura:** il Redattore produce o modifica il contenuto su branch dedicato;
2. **In Verifica:** il Redattore apre una Pull Request; il Verificatore controlla il documento e può approvare o richiedere modifiche;

3. **Verificato:** il documento ha superato la verifica tramite Pull Request;
4. **Approvato:** il documento verificato viene approvato dal Responsabile, eventualmente dopo review di gruppo;
5. **Rilasciato:** il documento approvato viene pubblicato nel repository e reso disponibile tramite sito documentale.

Un documento entra in baseline solo dopo approvazione del Responsabile e review di gruppo.

4.1.10 Versionamento Documentale

Il gruppo adotta il *Semantic Versioning_G* a tre cifre nel formato:

vX.Y.Z

Le versioni vengono incrementate secondo le seguenti regole:

Campo	Incremento	Quando Applicarlo
Major	+1.0.0	Ingresso in una nuova baseline o cambiamento strutturale profondo del documento.
Minor	+0.1.0	Aggiunta o modifica significativa di contenuti.
Patch	+0.0.1	Correzioni ortografiche, tipografiche o modifiche non semantiche.

L'incremento di versione è scelto dal Redattore, controllato dal Verificatore e approvato dal Responsabile in caso di rilascio ufficiale.

4.2 Processo di Gestione della Configurazione

4.2.1 Descrizione

Il processo di gestione della configurazione garantisce integrità, consistenza, versionamento e tracciabilità degli elementi configurativi del progetto.

Sono considerati elementi configurativi:

- documenti L^AT_EX;
- file PDF generati;
- template;
- asset grafici;
- script di automazione;
- codice del Proof of Concept, quando presente;
- configurazioni del repository;
- materiale pubblicato su GitHub Pages.

4.2.2 Sistema di Gestione

Il gruppo utilizza GitHub come sistema principale di gestione della configurazione.

Ogni modifica deve essere tracciata tramite:

- issue;
- branch dedicato;
- commit_G riferiti alla issue;
- Pull Request;
- review;
- merge_G controllato.

4.2.3 Struttura del Repository

A partire dallo Sprint 9, la repository ufficiale del progetto assume il nome **Second-Brain** e non è più destinata esclusivamente alla documentazione.

Essa contiene:

- documentazione di progetto;
- codice del Proof of Concept;
- script di automazione;
- configurazioni di supporto;
- asset condivisi;
- materiali pubblicati tramite GitHub Pages.

La struttura principale della repository è:

```
Second-Brain/  
+-- docs/  
+-- poc/  
|   +-- frontend/  
|   +-- backend/  
+-- scripts/
```

La cartella **docs/** contiene la documentazione di progetto.

La cartella **poc/** contiene il codice del Proof of Concept, separando frontend e backend.

La cartella **scripts/** contiene gli script di automazione già adottati dal gruppo, inclusi quelli a supporto del Glossario e del Piano di Qualifica.

Il sito GitHub Pages pubblica la documentazione e rende disponibile una versione compressa del PoC per il download.

La versione ufficiale e stabile di ogni elemento configurativo è quella presente nel branch **main_G**.

4.2.4 Gestione delle Issue

Le issue devono essere create esclusivamente tramite Google Forms.

A partire dallo Sprint 9, il form di creazione delle issue distingue tra attività documentali e attività non documentali.

4.2.4.1 Issue documentali Per le issue documentali continua a essere applicata la classificazione già adottata per la documentazione.

Il nome logico della issue documentale deve seguire la forma:

`doc_<categoria-uso>_<nome-documento>`

dove:

- `doc` identifica un'attività documentale;
- `categoria-uso` identifica la natura e la destinazione del documento, separando i due valori tramite trattino;
- `categoria` può assumere i valori `gestionale` o `tecnico`;
- `uso` può assumere i valori `interno` o `esterno`;
- `nome-documento` identifica il documento o l'artefatto documentale coinvolto, con parole separate da trattini.

Esempi:

- `doc_gestionale-interno_norme-di-progetto`;
- `doc_gestionale-esterno_piano-di-qualifica`;
- `doc_tecnico-esterno_analisi-dei-requisiti`.

Il sito documentale è considerato un artefatto documentale quando la modifica riguarda contenuti, collegamenti, indici o documenti pubblicati.

4.2.4.2 Issue non documentali Per le issue non documentali, il form distingue il tipo di attività secondo le seguenti categorie:

- **feature_G**: nuova funzionalità del prodotto o del PoC;
- **bug_G**: comportamento non corretto rispetto all'atteso;
- **refactor_G**: miglioramento interno senza modifica del comportamento osservabile;
- **test**: attività relativa ai test, prevista per le attività successive alla RTB.

La posizione deve essere scelta tra:

- **poc**;

- **frontend**;
- **backend**;
- **infra**;
- **test**.

Il nome logico della issue non documentale deve seguire la forma:

`<tipo>_<posizione>_<descrizione-breve>`

La separazione tra i campi principali avviene tramite il carattere `_`. La descrizione breve deve essere scritta in minuscolo, senza accenti e con parole separate da trattini.

Esempi:

- `feature_frontend_css`;
- `bug_frontend_css`;
- `refactor_infra_aggiornamento-token`;
- `test_backend.test-chiamata-ai`.

4.2.4.3 Campi obbligatori

Ogni issue deve contenere:

- tipo o categoria;
- posizione, uso o ambito;
- assegnatario;
- ruolo;
- priorità;
- scadenza;
- tempo previsto;
- milestone RTB o PB;
- descrizione dell'attività.

Le modifiche strutturali o configurative del sito documentale sono invece classificate come attività infrastrutturali con posizione **infra**.

4.2.5 Strategia di Branching

Il gruppo adotta un modello *trunk-based development*_G.

Il branch **main** è l'unico branch stabile. Ogni modifica deve essere sviluppata su branch dedicato, creato a partire dalla sezione *Development* della issue GitHub associata.

A partire dallo Sprint 9, con decorrenza dal 2026-06-02, il gruppo distingue tra attività documentali e attività non documentali.

Per tutte le nuove convenzioni di naming, il carattere `_` separa i campi principali, mentre il carattere `-` separa le parole interne ai singoli campi composti.

4.2.5.1 Branch documentali Per le attività documentali, la convenzione di naming dei branch è:

`<id>_doc_<categoria-uso>_<nome-documento>`

dove:

- `id` è l'identificativo della issue;
- `doc` identifica un'attività documentale;
- `categoria-uso` indica natura e destinazione del documento, separando i due valori tramite trattino;
- `categoria` può assumere i valori `gestionale` o `tecnico`;
- `uso` può assumere i valori `interno` o `esterno`;
- `nome-documento` identifica il documento o l'artefatto documentale modificato, con parole separate da trattini.

Esempi:

- `146_doc_gestionale-interno_norme-di-progetto`;
- `147_doc_gestionale-esterno_piano-di-qualifica`;
- `148_doc_tecnico-esterno_analisi-dei-requisiti`.

4.2.5.2 Branch non documentali Per le attività non documentali, la convenzione di naming dei branch è:

`<id>_<tipo>_<posizione>_<descrizione-breve>`

dove:

- `id` è l'identificativo della issue;
- `tipo` indica la natura dell'attività e può assumere i valori `feature`, `bug`, `refactor` o `test`;
- `posizione` indica l'area coinvolta e può assumere i valori `poc`, `frontend`, `backend`, `infra` o `test`;
- `descrizione-breve` sintetizza l'attività con parole minuscole, senza accenti e separate da trattini.

La separazione tra i campi principali avviene tramite il carattere `_`; la separazione tra le parole interne alla descrizione breve avviene tramite trattino.

Esempi:

- `142_feature_frontend-css`;

- 143_bug_frontend_css;
- 144_refactor_infra_aggiornamento-token;
- 145_test_backend_test-chiamata-ai.

Le issue e i branch già aperti prima dell'adozione della presente convenzione mantengono la nomenclatura originaria fino alla loro chiusura. Le nuove attività aperte a partire dallo Sprint 9 devono seguire le convenzioni aggiornate.

4.2.6 Commit

Ogni commit deve seguire una convenzione ispirata ai Conventional Commits_G, adattata al workflow del gruppo.

La forma prescritta è:

`<tipo>(<posizione>): <messaggio> (#id)`

dove:

- **tipo** identifica la natura della modifica;
- **posizione** indica l'area o il documento coinvolto;
- **messaggio** descrive sinteticamente la modifica;
- **id** è l'identificativo della issue collegata.

I tipi ammessi sono:

Tipo Task	Tipo Commit	Significato
feature	feat	Nuova funzionalità del prodotto o del PoC.
bug	fix	Correzione di un comportamento non corretto.
doc	docs	Modifica alla documentazione.
refactor	refactor	Miglioramento interno senza modifica del comportamento osservabile.
test	test	Aggiunta o modifica di test.

Il tipo del commit deve essere coerente con il tipo della issue associata (per le attività documentali, il tipo di commit è sempre **docs**).

Il messaggio deve essere scritto in italiano e in forma descrittiva.

Ogni commit deve riferirsi a una sola issue. Le Pull Request possono includere più issue solo quando le attività sono strettamente collaborative o tecnicamente indivisibili.

4.2.6.1 Scope Documentali Per i commit documentali devono essere utilizzati i seguenti scope_G:

- **ndp**: Norme di Progetto;
- **pdq**: Piano di Qualifica;
- **pdp**: Piano di Progetto;
- **adr**: Analisi dei Requisiti;
- **sa**: Analisi dello Stato dell'Arte;
- **glo**: Glossario;
- **viN**: Verbale interno numero N;
- **veN**: Verbale esterno numero N;
- **ddbN**: Diario di Bordo numero N;
- **sito**: sito documentale.

Lo scope **adr** indica l'Analisi dei Requisiti nel contesto dei commit documentali e non deve essere confuso con gli Architecture Decision Records.

4.2.6.2 Esempi

- `feat(frontend): aggiunto css (#142);`
- `fix(frontend): sistemato problema padding css (#143);`
- `docs(ndp): aggiunta sezione 2 (#144);`
- `refactor(infra): aggiornato token (#145);`
- `test(backend): aggiunto test chiamata ai (#146).`

4.2.7 Pull Request

Ogni modifica deve essere integrata tramite Pull Request.

La Pull Request deve:

- essere collegata alla issue corrispondente;
- descrivere sinteticamente le modifiche effettuate;
- essere aperta preferibilmente entro la domenica precedente la riunione del martedì;
- ricevere almeno una review positiva;
- non essere approvata dall'autore della modifica.

In caso di modifiche sostanziali richieste dal Verificatore, il Redattore può segnare la Pull Request come *draft* fino al completamento delle correzioni.

In fase PB, la *Pull Request Policy* viene rafforzata:

- È obbligatoria l'approvazione di almeno **1 reviewer** diverso dall'autore del codice.
- Il reviewer ha l'esplicito compito di verificare non solo la logica funzionale, ma anche il rigoroso rispetto delle regole di dipendenza tra gli strati architetturali.

4.2.8 Branch Protection Rules

Il branch `main` è protetto tramite branch protection rules_G.

Le regole attive prevedono:

- obbligo di almeno una review positiva;
- blocco dei force push_G;
- restrizione dell'eliminazione del branch protetto.

4.2.9 Merge

Il merge su `main` può essere eseguito solo dopo approvazione della Pull Request da parte del Verificatore.

Il merge viene eseguito dal Redattore, non dal Verificatore.

Il gruppo utilizza la modalità standard di merge proposta da GitHub tramite Pull Request.

Dopo il merge:

- la issue associata viene chiusa automaticamente;
- il branch di lavoro viene eliminato;
- il Redattore aggiorna il tempo effettivo_G nel foglio di consuntivazione.

4.2.10 Generazione e Pubblicazione dei PDF

I PDF vengono generati localmente tramite LaTeX Workshop.

Il sito GitHub Pages non viene aggiornato automaticamente a ogni merge su `main`; l'aggiornamento del sito documentale viene gestito tramite task periodica dedicata.

4.2.11 Gestione del Sito Documentale

Il sito documentale pubblicato tramite GitHub Pages costituisce un elemento configurativo del progetto e viene aggiornato mediante task dedicate.

Le attività relative al sito documentale sono distinte in due categorie:

- **Aggiornamenti documentali:** comprendono la pubblicazione o sostituzione di documenti PDF, l'aggiornamento degli indici, dei collegamenti, delle pagine informative e di ogni altro contenuto documentale reso disponibile sul sito.

Tali attività sono considerate documentali e devono seguire le convenzioni previste per documenti, issue e branch documentali.

Per queste attività deve essere utilizzato lo scope `sito` nei commit.

Esempi:

- branch: 215_doc_gestionale-esterno_sito-documentale;
- commit: docs(sito): aggiornato indice documenti (#215).

- **Aggiornamenti infrastrutturali:** comprendono modifiche alla struttura del sito, alla navigazione, al tema grafico, agli script di build, ai workflow GitHub Pages e alle configurazioni tecniche necessarie alla pubblicazione.

Tali attività sono considerate non documentali e devono essere classificate come attività di tipo **feature**, **bug** o **refactor** con posizione **infra**.

Esempi:

- branch: 216_feature_infra_navbar-sito;
- commit: feat(infra): aggiunta navbar sito (#216).

L'aggiornamento periodico del sito documentale finalizzato alla pubblicazione dei documenti approvati è considerato attività documentale.

4.3 Processo di Verifica

4.3.1 Descrizione

Il processo di verifica assicura che ogni artefatto prodotto sia conforme alle presenti norme e agli obiettivi di qualità definiti nel Piano di Qualifica.

La verifica risponde alla domanda:

Il prodotto o artefatto è stato realizzato correttamente rispetto alle norme stabilite?

In fase RTB e durante l'implementazione del Proof of Concept, la verifica riguarda:

- documentazione;
- coerenza dei riferimenti;
- correttezza delle matrici di tracciabilità;
- rispetto delle convenzioni;
- adeguatezza del materiale relativo al Proof of Concept;
- codice prodotto per il Proof of Concept;
- configurazioni necessarie all'esecuzione locale del PoC.

4.3.2 Responsabilità

- Il Verificatore deve essere sempre un membro diverso dal Redattore.
- Nessun membro può verificare e approvare il proprio lavoro.
- Il Verificatore non modifica direttamente l'artefatto verificato.
- Il Verificatore segnala le modifiche richieste tramite commenti o richieste di cambiamento nella Pull Request.
- Il Responsabile interviene nell'approvazione finale, non nella verifica puntuale dell'artefatto.

4.3.3 Attività di Verifica

Per i task documentali, il Verificatore deve controllare:

- correttezza linguistica;
- coerenza contenutistica;
- rispetto del template;
- correttezza del registro delle modifiche;
- correttezza dei riferimenti interni;
- funzionamento dei link;
- presenza del pedice_G dove necessario;
- coerenza con il Glossario;
- conformità alle convenzioni di naming;
- correttezza del versionamento;
- compilazione corretta del documento PDF;
- assenza di segnaposto non motivati.

Se la modifica coinvolge documenti collegati, il Verificatore deve controllare anche i riferimenti coinvolti.

In particolare, modifiche ai requisiti devono comportare il controllo di:

- Analisi dei Requisiti;
- matrici di tracciabilità;
- Glossario, se necessario;
- eventuali riferimenti nel Piano di Qualifica.

4.3.4 Verifica del Codice del Proof of Concept

Durante la fase RTB, la verifica del codice riguarda esclusivamente il codice prodotto per il Proof of Concept.

Per le Pull Request di implementazione, il Verificatore deve controllare:

- coerenza della modifica rispetto alla issue;
- correttezza della convenzione di branch e commit;
- avvio locale del PoC;
- assenza di errori evidenti in console;
- assenza di credenziali, token o file `.env` nel repository;

- aggiornamento del `READMEG` se la modifica cambia la procedura di avvio;
- descrizione della prova effettuata per modifiche backend o `APIG`;
- coerenza con requisiti e casi d'uso collegati, se disponibili.

La verifica del codice PoC non prevede test automatici obbligatori in fase RTB. Tali attività saranno normate quando il gruppo introdurrà il processo di testing nelle fasi successive.

4.3.5 Verifica del Codice (Analisi Statica e Testing)

In fase PB, il codice prodotto è soggetto ad analisi statica e testing automatico prima di poter essere integrato:

- **Analisi Statica (Linting/Formattazione):** `ruff` per il backend (Python) e la combinazione `eslint` + `prettier` per il frontend (TypeScript/Vue).
- **Testing Automatico:** `pytest` per il backend e `vitest` per il frontend.
- **Code Coverage:** È fissata una soglia minima di copertura dei test pari all'**80%**.

Tali controlli sono automatizzati tramite pipeline di Continuous Integration.

4.3.6 Esito della Verifica

La verifica può avere:

- **Esito positivo:** il Verificatore approva la Pull Request;
- **Esito negativo:** il Verificatore richiede modifiche tramite commenti o richiesta di cambiamento.

Una richiesta di modifica può bloccare il merge finché il Redattore non ha risolto le osservazioni.

4.3.7 Definition of Done

Un task è considerato *Done* solo se soddisfa i criteri comuni e, in base alla natura dell'attività, i criteri specifici per documentazione o implementazione.

4.3.7.1 Criteri Comuni

- il task è stato creato tramite `Google FormG`;
- il task è associato a una issue GitHub;
- la issue contiene tipo, posizione, assegnatario, ruolo, priorità, scadenza e tempo previsto;
- il task è stato sviluppato su branch dedicato;
- i commit rispettano la convenzione definita nelle presenti norme;
- la Pull Request è collegata alla issue corrispondente;

- la Pull Request ha ricevuto almeno una review positiva;
- il Verificatore è diverso dall'autore della modifica;
- il Redattore ha eseguito il merge;
- la issue è stata chiusa;
- il tempo effettivo è stato registrato.

4.3.7.2 Criteri per Task Documentali

- il contenuto è completo rispetto allo scope della issue;
- il registro delle modifiche è stato aggiornato, se necessario;
- il documento compila correttamente;
- il PDF è stato generato senza errori bloccanti;
- i link e i riferimenti interni sono corretti;
- il Glossario è stato aggiornato, se necessario;
- il pedice_G è stato applicato correttamente.

4.3.7.3 Criteri per Task di Implementazione del PoC

- il codice è coerente con lo scope della issue;
- il PoC si avvia localmente;
- non sono presenti errori evidenti in console;
- non sono presenti credenziali, token, file `.env` o dati sensibili nel repository;
- il README è aggiornato se la modifica cambia la procedura di avvio;
- l'Analisi dei Requisiti è aggiornata se emerge una modifica funzionale;
- l'Analisi dello Stato dell'Arte è aggiornata se cambia una tecnologia;
- il Piano di Qualifica è aggiornato se cambiano metriche o modalità di misurazione_G;
- il Glossario è aggiornato se vengono introdotti nuovi termini tecnici;
- per le issue di tipo **bug**, la Pull Request descrive la causa del problema e la correzione effettuata;
- per le issue di tipo **feature**, la Pull Request indica il requisito o il caso d'uso collegato, se disponibile.

4.3.7.4 Criteri per Task di Sviluppo (Fase PB) Oltre ai criteri comuni, un task di sviluppo del prodotto finale si considera completato solo se:

- il codice compila e non sono presenti errori rilevati dai tool di analisi statica (linting);
- i test di unità sono stati scritti e risultano eseguiti con successo;
- la *code coverage* del modulo modificato o aggiunto è $\geq 80\%$;
- la Pull Request è approvata da almeno un revisore, che ha controllato il rispetto dei layer architetturali;
- in caso di modifiche a interfacce condivise tra componenti, la modifica è stata supervisionata dall'architetto o referente di coerenza;
- la pipeline di Continuous Integration è passata con successo.

4.4 Processo di Validazione

4.4.1 Descrizione

Il processo di validazione ha lo scopo di accertare che quanto prodotto risponda alle esigenze della proponente e agli obiettivi del capitolato.

La validazione risponde alla domanda:

Stiamo costruendo il prodotto giusto?

In fase RTB, la validazione riguarda principalmente:

- requisiti;
- casi d'uso;
- scelte tecnologiche;
- perimetro del Proof of Concept.

4.4.2 Validazione con la Proponente

Ogni incontro di validazione con la proponente deve produrre un verbale esterno.

La issue relativa alla redazione del verbale esterno rimane aperta fino a quando il verbale non è stato firmato o confermato dalla proponente via email.

Se la proponente richiede modifiche rilevanti, queste devono essere:

- riportate nel verbale esterno;
- tradotte in issue dedicate;
- implementate tramite Pull Request;
- verificate dal Verificatore;
- integrate nei documenti coinvolti.

Le modifiche ai requisiti derivanti da feedback esterno devono aggiornare:

- Analisi dei Requisiti;
- matrici di tracciabilità;
- Glossario, se necessario;
- Piano di Qualifica, se impattano test, metriche o criteri di accettazione.

4.5 Processo di Accertamento della Qualità

4.5.1 Descrizione

Il processo di accertamento della qualità ha lo scopo di monitorare e migliorare la qualità dei processi e degli artefatti prodotti.

A differenza della verifica, che controlla puntualmente un artefatto, l'accertamento della qualità monitora l'efficacia e l'efficienza del way of working.

4.5.2 Modello di Miglioramento

Il gruppo adotta il ciclo PDCA_G come riferimento per il miglioramento continuo:

- **Plan:** definizione di obiettivi, metriche e procedure;
- **Do:** applicazione dei processi definiti;
- **Check:** misurazione dei risultati e analisi degli scostamenti;
- **Act:** introduzione di azioni correttive e aggiornamento del way of working.

4.5.3 Metriche e Piano di Qualifica

Il Piano di Qualifica definisce le metriche, le soglie e il cruscotto di qualità_G utilizzati dal gruppo per monitorare qualità di processo e qualità di prodotto.

Le Norme di Progetto disciplinano il processo di raccolta, aggiornamento e gestione delle metriche; formule, soglie, valori rilevati e rappresentazioni grafiche sono riportati nel [Piano di Qualifica](#).

L'Amministratore assegnato alla relativa task raccoglie i dati necessari, aggiorna il file di tracciamento e mantiene il cruscotto delle metriche.

Le metriche vengono rilevate e aggiornate al termine di ogni Sprint, integrando i dati raccolti nello sprint concluso e aggiornando le rappresentazioni tabellari e grafiche presenti nel Piano di Qualifica.

Le metriche relative al codice vengono calcolate solo sul codice mergiato nel branch **main**.

Se una metrica risulta fuori soglia, il gruppo ne discute durante la retrospective. L'esito della discussione viene verbalizzato e, qualora richieda una modifica operativa, viene creata una issue dedicata.

4.5.3.1 Metriche di qualità di processo Le metriche di qualità di processo misurano l'efficienza, la stabilità e la controllabilità del way of working adottato dal gruppo.

Il Piano di Qualifica definisce le seguenti metriche di processo:

Codice	Metrica	Finalità
MPC01	Schedule Performance Index _G (SPI _G)	Misurare l'efficienza nel rispetto dei tempi pianificati.
MPC02	Cost Performance Index _G (CPI _G)	Misurare l'efficienza nell'utilizzo del budget allocato.
MPC03	Requirements Stability Index _G (RSI _G)	Monitorare la stabilità dei requisiti e le variazioni introdotte nell'Analisi dei Requisiti.
MPC04	Time Efficiency _G	Misurare la percentuale di ore produttive rispetto al totale delle ore spese.
MPC05	Task Completion Rate _G	Misurare la percentuale di issue chiuse rispetto a quelle pianificate.
MPC06	PR Resolution Time	Misurare il tempo medio necessario per la revisione e il merge di una Pull Request.
MPC07	Review Coverage _G	Misurare la percentuale di Pull Request approvate da un revisore diverso dall'autore.
MPC08	Quality Metrics Compliance Rate _G	Misurare la percentuale di metriche di qualità il cui valore rilevato è almeno accettabile rispetto alle soglie definite.

4.5.3.2 Metriche di qualità di prodotto Le metriche di qualità di prodotto misurano la qualità degli artefatti prodotti, sia documentali sia software.

Il Piano di Qualifica definisce le seguenti metriche di prodotto:

Codice	Metrica	Finalità
MPD01	Soddisfacimento Requisiti Obbligatori	Misurare la percentuale di requisiti obbligatori implementati.
MPD02	Comment Density _G	Misurare il rapporto tra righe di commento e righe di codice totali.
MPD03	Browser Compatibility _G	Misurare il numero di browser target su cui il sistema è testato.

Continua nella pagina successiva

Codice	Metrica	Finalità
MPD04	Errori Ortografici	Misurare il numero di errori ortografici rilevati per pagina di documento.
MPD05	Complessità Ciclomatica _G	Misurare la complessità dei flussi di controllo delle funzioni del codice frontend e backend.
MPD06	Tempo Iniziale di Caricamento del Programma _G	Misurare il tempo necessario affinché l'interfaccia principale del PoC risulti disponibile e utilizzabile.
MPD07	Indice di Gulpease _G	Misurare la leggibilità dei documenti in lingua italiana.

Le formule, gli strumenti di dettaglio, le soglie e le modalità operative di misurazione sono riportati nel Piano di Qualifica.

4.6 Processo di Gestione dei Cambiamenti

4.6.1 Descrizione

Il processo di gestione dei cambiamenti disciplina la valutazione, l'approvazione, l'implementazione e la verifica delle modifiche rilevanti a documenti, requisiti, tecnologie e processi.

Ogni cambiamento rilevante deve essere tracciato.

4.6.2 Tipologie di Cambiamento

I cambiamenti vengono distinti in:

- **documentali:** modifiche a struttura o contenuto dei documenti;
- **di requisito:** modifiche a requisiti, casi d'uso o matrici di tracciabilità;
- **tecnologici:** modifiche allo stack o alle tecnologie candidate;
- **di processo:** modifiche al way of working o alle presenti norme.

4.6.3 Procedura

Ogni cambiamento segue la seguente procedura:

1. **Richiesta:** il cambiamento emerge da verbale interno, verbale esterno, retrospettiva, feedback della proponente o revisione esterna;
2. **Valutazione Di Impatto:** il gruppo valuta impatto su documenti, requisiti, tempi, costi, tecnologie o processi;
3. **Approvazione:** il cambiamento viene approvato dal ruolo competente;

4. **Implementazione:** viene creata una issue e il cambiamento viene implementato tramite branch e Pull Request;
5. **Verifica:** il Verificatore controlla la conformità della modifica;
6. **Chiusura:** la issue viene chiusa dopo merge e aggiornamento del tempo effettivo.

4.6.4 Responsabilità di Approvazione

- I cambiamenti documentali ordinari sono approvati tramite verifica della Pull Request.
- I cambiamenti ai requisiti sono approvati dall'Analista e dal Responsabile; se incidono sul perimetro funzionale, devono essere validati con la proponente.
- I cambiamenti tecnologici sono approvati dal Progettista e dal Responsabile; se rilevanti, devono essere discussi con la proponente.
- I cambiamenti alle Norme di Progetto sono approvati dall'Amministratore e dal Responsabile.

4.7 Processo di Gestione delle Non Conformità

4.7.1 Descrizione

Una non conformità è una deviazione rispetto alle presenti norme, al template, al Piano di Qualifica o agli accordi formalizzati nei verbali.

Le non conformità possono emergere durante:

- verifica tramite Pull Request;
- review di sprint;
- retrospettiva;
- revisione con committente;
- incontro con la proponente;
- controllo di un documento già rilasciato.

4.7.2 Classificazione

Le non conformità vengono classificate in:

- **Bloccante:** impedisce il merge, il rilascio o l'ingresso in baseline dell'artefatto;
- **Non Bloccante:** non impedisce il merge immediato, ma deve essere corretta tramite task tracciato nello sprint corrente o successivo.

4.7.3 Gestione

Se la non conformità viene individuata prima del merge, il Verificatore la segnala nella Pull Request tramite commento o richiesta di cambiamento.

Se la non conformità viene individuata dopo il rilascio, essa deve essere:

- verbalizzata nella riunione successiva;
- trasformata in task;
- corretta tramite issue dedicata;
- verificata tramite Pull Request;
- registrata nel changelog del documento, se comporta modifica documentale.

Una non conformità bloccante può impedire il rilascio di una baseline fino alla sua risoluzione.

5 Processi Organizzativi

In conformità allo standard ISO/IEC 12207:1995, i processi organizzativi comprendono le attività trasversali necessarie a governare il gruppo, mantenere l'infrastruttura di lavoro, favorire il miglioramento continuo e garantire l'acquisizione delle competenze necessarie.

Nel contesto del progetto *Second Brain*, e in accordo con il tailoring descritto nella Sezione 2, il gruppo *Synergo Unit* adotta i seguenti processi organizzativi:

- gestione;
- gestione dei rischi;
- comunicazione;
- infrastruttura;
- miglioramento;
- formazione.

5.1 Processo di Gestione

5.1.1 Descrizione

Il processo di gestione disciplina l'organizzazione interna del gruppo, la pianificazione delle attività, l'assegnazione e la rotazione dei ruoli, la conduzione degli sprint, il tracciamento dell'avanzamento e il monitoraggio di tempi e costi.

Esso fornisce la struttura organizzativa entro cui i processi primari e di supporto vengono attuati.

5.1.2 Composizione del Gruppo

Il gruppo *Synergo Unit* è composto da sei membri.

Ogni membro contribuisce allo svolgimento del progetto assumendo, nel tempo, i ruoli previsti dal Regolamento del Progetto Didattico.

La composizione del gruppo, la rotazione dei ruoli e la distribuzione delle ore sono riportate nel Piano di Progetto.

5.1.3 Ruoli del Gruppo

Il gruppo adotta i seguenti ruoli:

- Responsabile;
- Amministratore;
- Analista;
- Progettista;
- Programmatore_G;
- Verificatore.

Ogni membro può ricoprire un solo ruolo all'interno dello stesso sprint. Tuttavia, uno stesso ruolo può essere assegnato a più membri nello stesso sprint, qualora il carico di lavoro lo renda necessario.

In fase RTB, il ruolo di Programmatore viene attivato solo per attività di codifica limitate alla realizzazione del Proof of Concept. La piena attivazione del ruolo è prevista nella fase PB.

La rotazione dei ruoli è definita sprint per sprint durante la riunione di pianificazione. Essa tiene conto di:

- disponibilità temporale dei membri;
- carico di lavoro previsto;
- competenze richieste dalle attività;
- necessità di distribuire l'esperienza sui diversi ruoli;
- obiettivo di mantenere tutti i membri aggiornati sull'avanzamento del progetto.

Le assegnazioni sono decise collegialmente dal gruppo durante il planning e registrate nel verbale interno.

La rotazione dei ruoli e l'assegnazione delle attività devono favorire la distribuzione delle competenze all'interno del gruppo.

Il gruppo non adotta una logica di assegnazione basata esclusivamente sulla competenza pregressa del singolo membro: la scelta dell'assegnatario non deve ricadere automaticamente su chi possiede già maggiore familiarità con una specifica attività o tecnologia.

Quando una competenza è concentrata in uno o pochi membri, il gruppo deve favorire affiancamento, condivisione delle conoscenze e progressiva autonomia degli altri componenti, compatibilmente con scadenze, carico di lavoro e criticità dello sprint.

5.1.4 Costi Orari dei Ruoli

I costi orari adottati sono quelli previsti dal Regolamento del Progetto Didattico.

Ruolo	Costo Orario	Responsabilità Principale
Responsabile	30€/h	Coordina il gruppo, rappresenta il fornitore verso committente e proponente, approva il rilascio degli artefatti e supervisiona l'avanzamento complessivo.
Amministratore	20€/h	Assicura l'efficienza degli strumenti e delle procedure a supporto del way of working, mantiene le Norme di Progetto e gestisce l'infrastruttura operativa.
Analista	25€/h	Svolge attività di analisi dei requisiti, studio del dominio e confronto tra esigenze della proponente e funzionalità attese.

Continua nella pagina successiva

Ruolo	Costo Orario	Responsabilità Principale
Progettista	25 €/h	Svolge attività di progettazione, selezione tecnologica e definizione della soluzione tecnica.
Programmatore	15 €/h	Svolge attività di codifica. In fase PB implementa le funzionalità del Minimum Viable Product, scrive i test automatici e assicura che il codice rispetti le metriche di analisi statica.
Verificatore	15 €/h	Svolge attività di verifica sugli artefatti prodotti dagli altri membri, garantendo indipendenza rispetto al Redattore.

5.1.5 Responsabilità Operative Specifiche

Oltre alle responsabilità previste dal ruolo, il gruppo adotta le seguenti responsabilità operative:

- il **Responsabile** coordina le riunioni, approva i documenti verificati, supervisiona l'avanzamento complessivo e cura il coordinamento verso committente e proponente;
- l'**Amministratore** mantiene l'infrastruttura, gestisce Google Forms, Google Sheets, GitHub Pages, email istituzionale, verbali esterni e aggiornamenti delle Norme di Progetto;
- l'**Analista** cura l'Analisi dei Requisiti, le fonti, i casi d'uso, il tracciamento e il confronto con la proponente sugli aspetti funzionali;
- il **Progettista** cura la valutazione tecnica delle soluzioni, lo stack tecnologico e il perimetro tecnico del Proof of Concept;
- il **Programmatore** realizza codice solo quando necessario per il Proof of Concept in RTB e in modo più esteso nella fase PB;
- il **Verificatore** controlla gli artefatti prodotti da altri membri, senza modificare direttamente il lavoro verificato.

5.1.6 Pianificazione degli Sprint

Gli sprint hanno durata settimanale, con cadenza da martedì a martedì.

Le attività vengono pianificate durante la riunione del martedì, dopo la retrospettiva e la review dello sprint precedente.

Durante lo sprint planning il gruppo:

- individua le attività da svolgere;
- valuta priorità e scadenze;
- stima i tempi previsti;
- assegna i ruoli dello sprint;
- assegna i task ai membri;

- crea le issue tramite Google Form;
- aggiorna il backlog_G dello sprint.

I task vengono pianificati ordinariamente durante il planning del martedì. L'aggiunta di nuovi task durante lo sprint è ammessa solo in caso di necessità motivata e deve essere tracciata.

5.1.7 Gestione dei Task Non Completati

I task non completati entro la fine dello sprint non vengono chiusi e ricreati.

Essi devono essere:

- discussi durante la sprint review;
- motivati nel verbale interno;
- mantenuti aperti;
- riportati nella lista delle issue dello sprint in cui vengono posticipati.

Per i task non associati a modifiche della repository, la chiusura della issue spetta all'assegnatario, dopo il completamento dell'attività e la verifica in review.

5.1.8 Tracciamento Ore e Costi

Il tracciamento delle ore avviene in minuti.

Il tempo previsto viene inserito nel Google Form al momento della creazione del task. Il tempo effettivo viene inserito manualmente nel foglio Google condiviso:

- dopo il merge e la chiusura della issue, per attività con modifica della repository;
- al termine dell'attività, per task non associati a modifiche della repository.

Per ogni task devono essere registrati:

- tempo previsto del ruolo assegnatario;
- tempo effettivo del ruolo assegnatario;
- tempo previsto ed effettivo del Verificatore;
- tempo previsto ed effettivo del Responsabile, se coinvolto;
- tempo previsto ed effettivo dell'Amministratore, se coinvolto.

L'Amministratore controlla che i tempi effettivi siano compilati correttamente e li utilizza per:

- consuntivo_G di sprint;
- aggiornamento del Piano di Progetto;
- monitoraggio dei costi;

- analisi delle risorse residue_G.

Il monitoraggio dei costi deve garantire il rispetto del costo massimo accettato in fase di aggiudicazione. Qualora i costi sostenuti si avvicinino al limite previsto, il gruppo deve valutare azioni correttive e, se necessario, rinegoziare il perimetro dei requisiti con la proponente.

Il foglio di calcolo_G condiviso produce automaticamente:

- preventivo di sprint;
- consuntivo di sprint;
- delta ore;
- delta costi;
- risorse residue per ruolo;
- risorse residue per membro.

Le formule di conversione applicate sono:

$$\begin{aligned}\text{ore} &= \frac{\text{minuti}}{60} \\ \text{costo} &= \frac{\text{minuti}}{60} \times \text{costo orario del ruolo}\end{aligned}$$

5.1.9 Gestione dei Rischi

La gestione dei rischi è responsabilità del Responsabile e viene formalizzata nel Piano di Progetto.

Durante ogni sprint retrospective, il gruppo valuta:

- rischi occorsi;
- variazioni di probabilità o impatto dei rischi già identificati;
- nuovi rischi emersi;
- efficacia delle strategie di mitigazione_G adottate.

Gli aggiornamenti rilevanti vengono riportati nel Piano di Progetto e, se necessario, trasformati in azioni correttive o task operativi.

5.1.10 Riunione del Martedì

La riunione ordinaria del martedì si tiene alle 14:30 su Discord_G.

Qualora emergano impedimenti, l'orario può essere anticipato o posticipato previo accordo su WhatsApp_G. L'orario effettivo di inizio e fine viene riportato nel verbale interno.

La riunione ha durata variabile in base alla complessità delle decisioni da assumere.

La struttura della riunione è:

1. **Sprint Review:** il gruppo verifica lo stato dei task, analizza eventuali scostamenti tra preventivo e consuntivo e valuta i task non completati;
2. **Sprint Retrospective:** il gruppo analizza il processo seguito nello sprint precedente e individua almeno un'azione correttiva;
3. **Sprint Planning:** il gruppo pianifica lo sprint corrente, assegna ruoli e task e crea le relative issue.

Le decisioni, le azioni e le assegnazioni emerse vengono registrate nel documento di riunione condiviso e successivamente formalizzate nel verbale interno.

5.1.11 Riunione del Venerdì

La riunione del venerdì è fissata alle 14:30 su Discord e ha durata indicativa di 30 minuti.

La riunione può essere anticipata o posticipata, di comune accordo, al termine del Diario di Bordo, in modo da recepire tempestivamente eventuali aggiornamenti o osservazioni.

Essa ha lo scopo di:

- risolvere dubbi emersi durante lo sprint;
- affrontare criticità operative;
- aggiornare il gruppo sullo stato delle attività;
- recepire eventuali osservazioni emerse dal Diario di Bordo.

5.1.12 Daily Stand-Up Asincrono

Il gruppo utilizza WhatsApp per comunicazioni operative e coordinamento durante lo sprint.

Il Daily Stand-Up asincrono viene utilizzato quando utile per segnalare impedimenti, richieste di supporto, dubbi o aggiornamenti rilevanti rispetto alle attività in corso.

Lo stato di avanzamento delle attività non viene riportato sistematicamente tramite il Daily Stand-Up, poiché è tracciato in modo continuativo mediante GitHub Projects_G, issue_G, branch_G e pull request_G.

Quando necessario, i membri del gruppo possono comunque comunicare:

- avanzamenti significativi;
- attività previste nel breve termine;
- impedimenti;
- richieste di supporto.

5.1.13 Verbali e Tracciamento Decisioni

Ogni riunione interna produce un verbale interno.

Il verbale interno deve riportare:

- data, orario e piattaforma;
- partecipanti presenti e assenti;
- ordine del giorno;
- sintesi della discussione;
- decisioni prese;
- azioni da svolgere;
- riferimenti alle issue create;
- data della riunione successiva.

Le decisioni interne sono identificate tramite codice:

VI-y.x

dove y identifica il verbale e x la decisione progressiva.

Ogni decisione rilevante deve essere collegata, quando possibile, a una o più issue operative.

5.2 Processo di Comunicazione

5.2.1 Descrizione

Il processo di comunicazione disciplina i canali, le modalità e le responsabilità relative alle comunicazioni interne ed esterne del gruppo.

Una comunicazione corretta deve essere:

- tracciabile;
- tempestiva;
- pertinente;
- accessibile ai membri coinvolti;
- formalizzata quando produce decisioni o impegni.

5.2.2 Comunicazione Interna

I canali interni adottati sono:

- **Discord**: piattaforma principale per riunioni, discussioni operative e allineamenti sincroni;
- **WhatsApp**: canale per comunicazioni rapide, aggiornamenti asincroni, impedimenti e convocazioni straordinarie;
- **GitHub**: canale operativo per issue, Pull Request, review e tracciamento delle attività;
- **Google Calendar**: strumento per disponibilità, eventi ricorrenti e pianificazione delle riunioni;

- **Google Drive/Docs:** spazio di supporto per documenti di riunione condivisi e materiali temporanei.

Le decisioni prese su canali informali devono essere riportate nel primo verbale interno utile, qualora producano effetti su documenti, task, requisiti, strumenti o processi.

5.2.3 Comunicazione Esterna

Le comunicazioni esterne avvengono tramite l'email istituzionale del gruppo:

`synergounit.20@gmail.com`

L'indirizzo è gestito dall'Amministratore.

L'Amministratore cura la gestione operativa della casella email; il Responsabile mantiene la responsabilità del contenuto delle comunicazioni formali quando esse comportano impegni verso committente o proponente.

Le comunicazioni esterne devono passare attraverso l'email istituzionale, salvo diversa indicazione esplicita della proponente o del committente.

5.2.4 Incontri con la Proponente

Gli incontri con la proponente avvengono tramite Google Meet.

Il processo prevede:

1. durante una riunione interna il gruppo individua i temi da discutere;
2. il gruppo definisce l'ordine del giorno;
3. l'Amministratore contatta la proponente tramite email ufficiale;
4. dopo conferma della disponibilità, il gruppo invia l'evento Google Meet;
5. prima dell'incontro vengono definiti i portavoce per i diversi argomenti;
6. al termine dell'incontro l'Amministratore redige il verbale esterno;
7. il verbale viene inviato alla proponente per firma;
8. dopo conferma, il verbale viene caricato, verificato, approvato e pubblicato.

Il portavoce dell'incontro viene scelto in base all'argomento trattato. In caso di più tematiche, il gruppo stabilisce preventivamente chi interviene su ciascun punto, mantenendo comunque la possibilità per ogni membro di porre domande.

L'obiettivo è ottimizzare il tempo dell'incontro, evitare tempi morti e garantire che tutti i dubbi preparati vengano affrontati.

5.2.5 Comunicazione con il Committente

La comunicazione con il committente avviene tramite:

- email istituzionale del gruppo;
- Diari di Bordo;
- revisioni di avanzamento;
- incontri o ricevimenti richiesti.

Gli esiti rilevanti delle comunicazioni con il committente devono essere analizzati dal gruppo e, se necessario, tradotti in azioni operative.

5.3 Processo di Infrastruttura

5.3.1 Descrizione

Il processo di infrastruttura definisce e mantiene l'insieme degli strumenti utilizzati dal gruppo per supportare pianificazione, comunicazione, documentazione, sviluppo, pubblicazione e tracciamento.

L'infrastruttura deve essere mantenuta coerente con i processi definiti nelle presenti Norme di Progetto.

5.3.2 Responsabilità

L'Amministratore è responsabile della manutenzione dell'infrastruttura operativa.

In particolare, l'Amministratore:

- mantiene configurati gli strumenti di gestione;
- mantiene Google Forms e Google Sheets;
- gestisce l'email istituzionale;
- aggiorna il sito GitHub Pages;
- supporta i membri nell'utilizzo degli strumenti;
- propone aggiornamenti alle Norme di Progetto quando cambia l'infrastruttura.

5.3.3 Strumenti di Gestione

Gli strumenti di gestione adottati sono:

- **GitHub Projects:** pianificazione e tracciamento dello stato delle issue;
- **Google Forms:** creazione standardizzata dei task e generazione automatica delle issue;
- **Google Sheets:** raccolta di stime, consuntivi e calcolo automatico di ore, costi e risorse residue;
- **Google Calendar:** gestione di riunioni, disponibilità ed eventi.

5.3.4 Strumenti di Documentazione

Gli strumenti di documentazione adottati sono:

- **L^AT_EX**;
- **Visual Studio Code**;
- **LaTeX Workshop**;
- **Script Python per Glossario**;
- **GitHub Pages**;
- **Canva**.

Le modalità operative di utilizzo di tali strumenti sono descritte nel Processo di Documentazione e nel Processo di Gestione della Configurazione.

5.3.5 Strumenti di Comunicazione

Gli strumenti di comunicazione adottati sono:

- **Discord**;
- **WhatsApp**;
- **Email istituzionale**;
- **Google Meet**;
- **Google Drive/Docs**.

5.3.6 Strumenti Tecnici

Gli strumenti e le tecnologie tecniche adottati per il Proof of Concept includono:

- **Vue.js**;
- **CodeMirror**;
- **Python**;
- **FastAPI**;
- **Docker**;
- **Docker Compose**;
- **GitHub**;
- **LucidSpark con elementi UML di LucidChart**.
- **StarUML**

In fase RTB il Proof of Concept non prevede persistenza su database. L'eventuale adozione di PostgreSQL rimane collegata alle successive decisioni progettuali e sarà rivalutata nella fase PB, in base agli esiti del Proof of Concept e al confronto con la proponente.

Le decisioni definitive relative allo stack tecnologico sono riportate nell'Analisi dello Stato dell'Arte e potranno essere aggiornate in seguito al confronto con la proponente e agli esiti del Proof of Concept.

5.3.7 Infrastruttura del Proof of Concept

L'infrastruttura del Proof of Concept è contenuta nella cartella **poc/** della repository **Second-Brain**.

Il PoC viene eseguito tramite Docker Compose e comprende due servizi:

- **frontend**: servizio dedicato all'interfaccia Vue.js;
- **backend**: servizio dedicato alle API FastAPI.

La procedura di avvio del PoC deve essere documentata nel README della cartella **poc/**.

Il README deve contenere almeno:

- prerequisiti;
- procedura di avvio tramite Docker Compose;
- porte utilizzate dai servizi;
- modalità di verifica dell'avvio corretto;
- eventuali problemi noti.

Il README tecnico del PoC è mantenuto dal Progettista quando riguarda scelte tecniche o architetturali, e dal Programmatore quando riguarda comandi operativi, setup ed esecuzione locale.

5.3.8 Struttura del Repository di Sviluppo

In fase PB, il codice sorgente è organizzato all'interno del repository in due macro-cartelle principali, al fine di garantire una netta separazione delle responsabilità tra i due domini.

5.3.8.1 Frontend (Vue.js / TypeScript) L'applicazione lato client è organizzata secondo il pattern MVC. La struttura interna del package, situata all'interno della cartella **src/**, è la seguente:

- **model/**: contiene lo stato applicativo e la logica dei dati;
- **view/**: contiene i componenti grafici e di presentazione Vue;
- **controller/**: gestisce e smista gli eventi generati dall'utente;
- **proxy/**: implementa le interfacce di servizio traducendo le chiamate verso il backend;
- **bootstrap/**: gestisce la composizione manuale e l'iniezione delle dipendenze.

5.3.8.2 Backend (FastAPI / Python) L'applicazione lato server adotta una *Layered Architecture*. La struttura interna dei package, situata all'interno della cartella **app/**, è la seguente:

- **api/**: *Presentation Layer* per l'esposizione degli endpoint REST e la validazione;
- **service/**: *Business Layer* per l'orchestrazione dei casi d'uso;
- **domain/**: contiene le entità di dominio, i modelli e le operazioni AI;
- **llm/**: contiene le interfacce e gli adapter per comunicare con i provider LLM;
- **repository/**: *Persistence Layer* dedicato all'accesso e al salvataggio dei dati;
- **export/**: gestisce le implementazioni per i vari formati di esportazione;
- **di/**: contiene i provider per la Dependency Injection.

5.3.9 Adozione di Nuovi Strumenti

L'adozione di un nuovo strumento deve seguire la seguente procedura:

1. proposta durante una riunione interna;
2. valutazione collegiale di utilità, costi, benefici e impatto sul workflow;
3. approvazione del gruppo;
4. registrazione della decisione nel verbale interno;
5. aggiornamento delle Norme di Progetto prima dell'utilizzo operativo.

5.4 Processo di Miglioramento

5.4.1 Descrizione

Il processo di miglioramento ha lo scopo di individuare inefficienze nei processi attivi e introdurre azioni correttive finalizzate al miglioramento continuo del way of working.

Il processo è collegato al ciclo PDCA descritto nella Sezione 4.5 e viene attuato principalmente tramite retrospective, consuntivi di sprint e aggiornamento delle presenti Norme di Progetto.

5.4.2 Identificazione delle Criticità

Le criticità possono emergere da:

- sprint review;
- sprint retrospective;
- scostamenti tra preventivo e consuntivo;
- difficoltà operative riportate dai membri;
- non conformità;
- feedback di committente o proponente;
- problemi nell'utilizzo degli strumenti.

5.4.3 Azioni Correttive

Ogni sprint retrospective deve produrre almeno una azione correttiva, anche minima.

Le azioni correttive devono essere:

- verbalizzate nel verbale interno;
- riportate nel consuntivo di sprint;
- monitorate nello sprint successivo;
- trasformate in aggiornamenti delle Norme di Progetto, se modificano il way of working.

Le azioni correttive possono riguardare:

- processo;
- comunicazione;
- strumenti;
- pianificazione;
- qualità documentale;
- gestione dei task;
- stima e consuntivazione.

5.4.4 Aggiornamento del Way of Working

Nessun processo può essere modificato unilateralmente.

Ogni modifica al way of working deve:

- essere discussa in riunione;
- essere approvata dal gruppo;
- essere verbalizzata;
- essere recepita nelle Norme di Progetto dall'Amministratore.

5.5 Processo di Formazione

5.5.1 Descrizione

Il processo di formazione ha lo scopo di garantire che i membri del gruppo acquisiscano le competenze necessarie per utilizzare strumenti, tecnologie e processi adottati.

La formazione riduce i rischi derivanti da inesperienza tecnica, rotazione dei ruoli e introduzione di nuovi strumenti.

5.5.2 Modalità di Formazione

La formazione viene pianificata tramite task specifici quando una nuova tecnologia, uno strumento o una procedura richiede studio preliminare.

Le attività di formazione possono avvenire tramite:

- studio individuale della documentazione ufficiale;
- prove pratiche su repository o ambienti locali;
- confronto tra membri;
- sessioni di allineamento su Discord;
- condivisione di indicazioni operative da parte di membri più esperti.

Le ore dedicate alla formazione sono considerate ore di palestra e non vengono consuntivate come ore produttive di progetto.

5.5.3 Formazione su Strumenti di Processo

La formazione sugli strumenti di processo è coordinata dall'Amministratore.

Essa riguarda principalmente:

- GitHub Projects;
- Google Forms;
- Google Sheets;
- GitHub Pages;
- template documentali;
- script di automazione del Glossario.

5.5.4 Formazione Tecnica

La formazione tecnica è coordinata dai membri che ricoprono i ruoli di Analista e Progettista, in base all'ambito trattato.

Essa riguarda principalmente:

- Vue.js;
- CodeMirror;
- Python;
- FastAPI;
- PostgreSQL, qualora venga confermato nelle successive decisioni progettuali;
- Docker;

- strumenti UML;
- strumenti di sviluppo collegati al Proof of Concept.

Le risorse di riferimento principali sono le documentazioni ufficiali delle tecnologie adottate.

5.5.5 Applicazione della Formazione

La formazione deve essere avviata prima dell'utilizzo operativo dello strumento o della tecnologia.

Qualora l'apprendimento possa avvenire in modo incrementale, il membro può proseguire la formazione durante l'attività, purché siano garantiti:

- comprensione minima dello strumento;
- supporto da parte del gruppo;
- verifica dell'artefatto prodotto;
- tracciabilità dell'attività svolta.

A Glossario

Per favorire la comprensione del documento e del progetto in generale, il gruppo ha redatto un file esterno dedicato al [Glossario](#). All'interno di quel documento sono definiti tutti gli acronimi, i termini tecnici, le tecnologie e i concetti di dominio propri dell'azienda proponente che potrebbero risultare ambigui per un lettore esterno.